FONOKIDS

Pipeline de audio y video

Un recorrido por cada punto, con las funciones que intervienen, el archivo y la línea. Tocá una fase para ver el detalle.

1. Extracción (video y audio) → 2. Modelos y GOP

Tu modelo entrenado | Modelo externo ya entrenado | Código / procedimiento

Las líneas corresponden al código actual. Si editás un archivo se corren, y el nombre de la función sirve para volver a encontrarla con Ctrl+F.

FASE 2A

Extracción de video

Del video a los 68 puntos de la cara. Los mismos pasos corren dos veces: al preparar el dataset y en vivo en el servidor.

ENTRA

Video con T frames

SALE

Entrenamiento: .npy de (T, 120) por clip · En vivo: tensor (1, 60, 120) y máscara (1, 60)

Puntos de boca: **índices 48 a 67 (main:82)**

Salto máximo: **0.15 (extraer:252 · main:168)**

Suavizado alpha: **0.4 (extraer:294 · main:226)**

Detector: **hrnet (extraer:14)**

Lote de HRNet: **4 frames (main:83)**

Ancho máximo: **640 (main:55)**

Recorrido (mismo en entrenamiento y en vivo)

1 Detector HRNet / FAN

Modelo externo ya entrenado

entrenamiento `scripts/extraer_landmarks_npy.py:178`

en vivo `server/main.py:89`

`_build_hrnet_detector()` · `build_hrnet_detector()`

Paquete face-alignment, modelo ya entrenado. Se crea una vez. En el servidor se carga al arrancar (línea 573).

```
  364  def _build_hrnet_detector():
  365      """Detector tipo HRNet: usa el paquete `face-alignment` (red
FAN, la
  366      implementación GPU/PyTorch de referencia para landmarks
faciales de
  367      alta fidelidad -- no es literalmente el repo original de
HRNet-Facial-
  368      Landmark-Detection, que requiere clonar su código y bajar
pesos de
  369      links externos inestables; esta es la vía estable/instalable
con pip
  370      que da resultados del mismo nivel académico)."""
  371      import face_alignment
  372      import torch
  373      import torch._dynamo
  374
  375      # face_alignment intenta compilar su red con torch.compile
(backend
  376      # "inductor") en la primera llamada. En Windows no hay build
oficial de
  377      # Triton, así que esa compilación SIEMPRE falla -- sin esto,
la falla
  378      # tira una excepción en vez de caer a modo eager (que funciona
perfecto,
```

```
  151  def build_hrnet_detector():
  152      """Detector tipo HRNet vía el paquete `face-alignment` (red
FAN)."""
  153      import face_alignment
  154      import torch._dynamo
  155
  156      # face_alignment intenta compilar su red con torch.compile en
la primera
  157      # llamada -- en Windows no hay build oficial de Triton, así
que esa
  158      # compilación SIEMPRE falla. Sin esto tira una excepción en
vez de caer
  159      # a modo eager (que funciona perfecto, solo un poco más lento
la primera vez).
  160      torch._dynamo.config.suppress_errors = True
```

```
  161
  162       if DEVICE == "cpu":
  163           print("[AVISO] No hay GPU disponible -- HRNet/FAN en CPU
va a ser MUY lento.")
  164
  165       # compile=False: face-alignment 1.5.0 compila su red con
torch.compile
  166       # por defecto -- en esta PC (Windows, sin Triton) ese warm-up
```

↓

2 Landmarks por frame

Modelo externo ya entrenado

entrenamiento `scripts/extraer_landmarks_npy.py:110`

en vivo `server/main.py:101`

`detect_frame_landmarks(landmarker, frame_bgr, timestamp_ms, frame_ms=40)` · `detect_landmarks_batch(detector, frames_bgr)`

Da 68 puntos (x, y) por frame, o None si no encuentra cara. En vivo procesa de a 4 frames.

```
  276  def detect_frame_landmarks(landmarker, frame_bgr, timestamp_ms,
frame_ms=40):
  277      """Detecta landmarks en UN frame (BGR, como lo da
OpenCV/cv2.VideoCapture).
  278
  279      Devuelve (landmarks_px, next_timestamp_ms):
  280          landmarks_px: array (N, 2) en píxeles, o None si no
detectó cara.
  281                        N y el significado de cada índice dependen
de
  282                        DETECTOR_BACKEND (ver
LIP_INDICES/MOUTH_RIGHT_CORNER/
  283                        MOUTH_LEFT_CORNER).
  284          next_timestamp_ms: para MediaPipe hay que seguir pasándolo
al
  285                        siguiente frame (exige timestamps
crecientes); HRNet
  286                        no lo necesita pero se devuelve igual para
que el
  287                        código que llama no tenga que saber qué
backend está
  288                        activo.
  289      """
  290      if DETECTOR_BACKEND == "hrnet":
```

```
  177  def detect_landmarks_batch(detector, frames_bgr):
  178      """Detecta landmarks en un clip entero: detección de cara en
sub-lotes
  179      (GPU), regresión de los 68 puntos frame por frame -- ver el
porqué de
  180      los sub-lotes en el comentario de HRNET_BATCH_CHUNK_SIZE."""
  181      if not frames_bgr:
  182          return []
  183
  184      frames_rgb = [cv2.cvtColor(f, cv2.COLOR_BGR2RGB) for f in
frames_bgr]
  185      device = detector.face_detector.device
  186      chunk_size = HRNET_BATCH_CHUNK_SIZE
  187      detected_faces = []
```

```
  188      try:
  189          for start in range(0, len(frames_rgb), chunk_size):
  190              chunk = frames_rgb[start:start + chunk_size]
  191              batch = torch.stack([
  192
torch.from_numpy(np.ascontiguousarray(f)).permute(2, 0, 1) for f in chunk
  193              ]).to(device)
  194              try:
  195                  chunk_faces =
```

↓

3 normalize_lip_landmarks

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:230`

en vivo `server/main.py:145`

```
def normalize_lip_landmarks(landmarks_px)
```

Toma los 20 puntos de la boca. Centra en la boca, rota con las comisuras y escala por su ancho. Devuelve 40 números.

```
  444  def normalize_lip_landmarks(landmarks_px):
  445      """landmarks_px: array (478, 2) en pixeles. Devuelve un vector
(80,)
  446      con los 40 puntos de labios normalizados
(traslación+rotación+escala),
  447      o None si algo sale mal (ej. comisuras casi superpuestas ->
escala ~0).
  448
  449      Normaliza contra las COMISURAS DE LA BOCA (antes se usaba la
distancia
  450      entre los ojos) -- pedido explícito: si la cámara queda
encuadrada muy
  451      cerca (algo muy común grabando en selfie), los ojos se salen
del cuadro
  452      y el detector de cara fallaba en el 90%+ de los frames, aunque
la boca
  453      se viera perfecto. Usando la boca misma como referencia, el
encuadre de
  454      los ojos deja de importar -- solo hace falta que la boca esté
visible,
  455      que es lo único que en verdad usa el modelo."""
  456      right_corner = landmarks_px[MOUTH_RIGHT_CORNER]
  457      left_corner = landmarks_px[MOUTH_LEFT_CORNER]
  458
```

```
  227  def normalize_lip_landmarks(landmarks_px):
  228      """landmarks_px: array (68, 2) en píxeles (esquema HRNet).
Devuelve un
  229      vector (40,) con los 20 puntos de labios normalizados
(traslación +
  230      rotación + escala), o None si algo sale mal (ej. comisuras
casi
  231      superpuestas). Normaliza contra las COMISURAS DE LA BOCA, no
los ojos
  232      -- ver el comentario largo en extraer_landmarks_npy.py sobre
por qué."""
  233      right_corner = landmarks_px[MOUTH_RIGHT_CORNER]
  234      left_corner = landmarks_px[MOUTH_LEFT_CORNER]
  235
  236      mouth_center = (right_corner + left_corner) / 2.0
```

```
  237      mouth_vector = left_corner - right_corner
  238      scale = np.linalg.norm(mouth_vector)
  239      if scale < 1e-3:
  240          return None
  241      angle = math.atan2(mouth_vector[1], mouth_vector[0])
  242
  243      cos_a, sin_a = math.cos(-angle), math.sin(-angle)
  244      rotation = np.array([[cos_a, -sin_a], [sin_a, cos_a]],
dtype=np.float64)
```

↓

4

is_outlier

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:255`

en vivo `server/main.py:187`

```
def is_outlier(vector, last_valid_vector)
```

Si algún valor salta más de 0.15 respecto al frame anterior, ese frame se trata como sin detección.

```
  487  def is_outlier(vector, last_valid_vector):
  488      """True si `vector` saltó de forma implausible respecto al
último frame
  489      válido -- indica un glitch de detección, no movimiento real."""
  490      if last_valid_vector is None:
  491          return False
  492      return float(np.abs(vector - last_valid_vector).max()) >
OUTLIER_MAX_JUMP
```

```
  257  def is_outlier(vector, last_valid_vector):
  258      if last_valid_vector is None:
  259          return False
  260      return float(np.abs(vector - last_valid_vector).max()) >
OUTLIER_MAX_JUMP
```

↓

5 fill_missing_frames

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:261`

en vivo `server/main.py:192`

```
def fill_missing_frames(positions)
```

Rellena los frames sin cara: copia al inicio, interpolación lineal en el medio y copia al final.

```
  498  def fill_missing_frames(positions):
  499      """positions: lista de longitud T, cada elemento es un array
(80,) o
  500      None (frame sin cara detectada). Devuelve (T, 80) SIN ningún
None:
  501
  502          - bfill: los frames iniciales sin detección se rellenan
con la
  503            primera posición válida (no se desfasa el tiempo, y no
hay
  504            ningún frame "anterior" real con el que interpolar).
  505          - Huecos EN EL MEDIO (con un frame válido antes y otro
después):
  506            interpolación LINEAL entre esos dos puntos -- más
preciso que
  507            copiar un solo valor, porque asume que la boca se movió
gradual
  508            entre A y B, en vez de quedarse congelada.
  509          - ffill: los huecos al FINAL (sin ningún frame válido
después) se
  510            rellenan con la última posición válida -- no hay
"después" real
  511            con el que interpolar, copiar es lo único posible.
  512
```

```
  263  def fill_missing_frames(positions):
  264      """positions: lista de (40,) o None por frame. Devuelve (T,
40) sin
  265      ningún None -- bfill al principio, interpolación lineal en
huecos del
  266      medio, ffill al final. None si NINGÚN frame tuvo posición
válida."""
  267      primeros_validos = [i for i, p in enumerate(positions) if p is
not None]
  268      if not primeros_validos:
  269          return None
  270
  271      primer_valido = primeros_validos[0]
  272      ultimo_valido_idx = primeros_validos[-1]
  273      relleno = list(positions)
```

```
274
275     for i in range(primer_valido):
276         relleno[i] = relleno[primer_valido]
277
278     i = primer_valido
279     while i < ultimo_valido_idx:
280         if relleno[i + 1] is None:
281             j = i + 1
```

↓

6

smooth_positions

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:297`

en vivo `server/main.py:229`

```
def smooth_positions(positions, alpha=SMOOTHING_ALPHA)
```

Media móvil exponencial con alpha 0.4 para quitar el temblor.

```
556 def smooth_positions(positions, alpha=SMOOTHING_ALPHA):
557     """Suavizado exponencial (EMA) sobre la posición YA continua
(sin None,
558     ver fill_missing_frames), aplicado ANTES de calcular velocidad/
559     aceleración -- derivar amplifica el ruido/jitter del detector,
así que
560     suavizar la posición primero da una dinámica mucho más
limpia."""
561     smoothed = np.empty_like(positions)
562     smoothed[0] = positions[0]
563     for t in range(1, positions.shape[0]):
564         smoothed[t] = alpha * positions[t] + (1 - alpha) *
smoothed[t - 1]
565     return smoothed.astype(np.float32)
```

```
302 def smooth_positions(positions, alpha=SMOOTHING_ALPHA):
303     smoothed = np.empty_like(positions)
304     smoothed[0] = positions[0]
305     for t in range(1, positions.shape[0]):
306         smoothed[t] = alpha * positions[t] + (1 - alpha) *
smoothed[t - 1]
307     return smoothed.astype(np.float32)
```

↓

7 add_dynamics

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:305`

en vivo `server/main.py:236`

```
def add_dynamics(positions)
```

(T, 40) pasa a (T, 120): posición, velocidad y aceleración.

```
  571  def add_dynamics(positions):
  572      """positions: (T, 80), ya continuo (sin None, ver
fill_missing_frames).
  573      Devuelve (T, 240) = concat([posición, velocidad, aceleración],
eje=1).
  574
  575      Delta1[0] y Delta2[0] (primer frame) se definen como cero -- no
hay
  576      frame anterior con el que calcular una diferencia."""
  577      velocity = np.zeros_like(positions)
  578      velocity[1:] = positions[1:] - positions[:-1]  # Delta1 = pos[t]
- pos[t-1]
  579
  580      acceleration = np.zeros_like(positions)
  581      acceleration[1:] = velocity[1:] - velocity[:-1]  # Delta2 =
Delta1[t] - Delta1[t-1]
  582
  583      return np.concatenate([positions, velocity, acceleration],
axis=1).astype(np.float32)  # (T, 240)
```

```
  310  def add_dynamics(positions):
  311      """(T, 40) -> (T, 120) = [posición | velocidad |
aceleración]."""
  312      velocity = np.zeros_like(positions)
  313      velocity[1:] = positions[1:] - positions[:-1]
  314
  315      acceleration = np.zeros_like(positions)
  316      acceleration[1:] = velocity[1:] - velocity[:-1]
  317
  318      return np.concatenate([positions, velocity, acceleration],
axis=1).astype(np.float32)
```

↓

8 Cierre del recorrido

Código / procedimiento

entrenamiento `scripts/extraer_landmarks_npy.py:315`

en vivo `server/main.py:549`

`process_clip(landmarker, video_path, start_timestamp_ms)` · `preprocess_sequence(positions, max_frames)`

Entrenamiento: process_clip hace todos los pasos sobre un clip. En vivo: preprocess_sequence suaviza, agrega dinámica, recorta del medio o rellena a 60 frames y arma la máscara.

```
  586  def process_clip(landmarker, video_path, start_timestamp_ms):
  587      """Procesa un clip entero preservando la longitud T EXACTA (un
frame
  588      del video = un frame de salida, sin saltear ni desfasar
tiempo).
  589
  590      Devuelve (sequence, detected, total, next_timestamp_ms):
  591          sequence: (T, 240) float32, o None si NINGÚN frame detectó
cara
  592                    (clip descartable).
  593          detected: cantidad de frames donde SÍ se detectó cara
(antes de
  594                    rellenar) -- para reportar calidad de la toma.
  595          total: cantidad total de frames del clip.
  596
  597      El timestamp tiene que seguir subiendo durante toda la vida
del
  598      landmarker (no reiniciarse por clip), por eso se recibe y se
devuelve.
  599      """
  600      cap = cv2.VideoCapture(video_path)
  601      fps = cap.get(cv2.CAP_PROP_FPS) or 25
  602      frame_ms = max(int(1000 / fps), 1)
  603
```

```
  641  def preprocess_sequence(positions, max_frames):
  642      """Copiado de scripts/probar_modelo.py -- suaviza, agrega
velocidad/
  643      aceleración, y padea/recorta a max_frames. Recorte del MEDIO
(no del
  644      principio) si la toma duró más que max_frames -- ver el porqué
en
  645      probar_modelo.py. NO se remuestrea por interpolación: este
modelo usa
  646      derivadas frame-a-frame (velocidad/aceleración), que se
romperían."""
  647      smoothed = smooth_positions(positions)
  648      sequence_np = add_dynamics(smoothed)
```

```
  649      sequence = torch.from_numpy(sequence_np).float()
  650
  651      T = sequence.shape[0]
  652      if T < max_frames:
  653          real_length = T
  654          padding = torch.zeros((max_frames - T,) +
sequence.shape[1:], dtype=sequence.dtype)
  655          sequence = torch.cat([sequence, padding], dim=0)
  656      elif T > max_frames:
  657          start = (T - max_frames) // 2
  658          sequence = sequence[start:start + max_frames]
```

Solo entrenamiento (extraer_landmarks_npy.py)

1

main

Código / procedimiento

`scripts/extraer_landmarks_npy.py:406`

```
def main()
```

Recorre las clases, procesa cada clip pendiente, guarda el .npy (línea 785) y actualiza label_map.json (línea 740).

```
  709  def main():
  710      words = find_words_to_process()
  711      if not words:
  712          print(f"No hay ninguna palabra grabada en {RUTA_CLIPS}.")
  713          return
  714
  715      print(f"Palabras a procesar ({len(words)}): {words}")
  716      print(f"Dimensión de salida por frame: {LANDMARK_DIM} "
  717            f"(posición {POSITION_DIM} + velocidad {POSITION_DIM} +
aceleración {POSITION_DIM})")
  718      landmarker = build_landmarker()
  719      label_map = {}
  720      if os.path.exists(LABEL_MAP_PATH):
  721          with open(LABEL_MAP_PATH, "r", encoding="utf-8") as f:
  722              label_map = json.load(f)
  723
  724      timestamp_ms = 0
  725      total_ok, total_fail = 0, 0
  726
  727      for word in words:
  728          clips_dir = os.path.join(RUTA_CLIPS, word, "clips")
  729          if not os.path.isdir(clips_dir):
  730              print(f"[{word}] no encontré {clips_dir}, se omite.")
  731              continue
```

↓

2 **find_words_to_process** Código / procedimiento

`scripts/extraer_landmarks_npy.py:397`

`def find_words_to_process()`

Si le pasás una clase por argumento procesa solo esa; si no, procesa todas las carpetas.

```
  700  def find_words_to_process():
  701      if len(sys.argv) >= 2:
  702          return [sys.argv[1].strip().lower()]
  703      if not os.path.exists(RUTA_CLIPS):
  704          return []
  705      return sorted(d for d in os.listdir(RUTA_CLIPS)
  706                    if os.path.isdir(os.path.join(RUTA_CLIPS, d)))
```

↓

3 **build_landmarker** Código / procedimiento

`scripts/extraer_landmarks_npy.py:52`

`def build_landmarker()`

Elige el detector según DETECTOR_BACKEND (línea 82). Hoy es hrnet.

```
  163  def build_landmarker():
  164      """Construye el detector según DETECTOR_BACKEND. Devuelve un
objeto
  165      opaco -- pasarlo tal cual a detect_frame_landmarks()."""
  166      if DETECTOR_BACKEND == "hrnet":
  167          return _build_hrnet_detector()
  168      return _build_mediapipe_landmarker()
```

Solo en vivo (validaciones del servidor)

1 Lectura de frames

Código / procedimiento

`server/main.py:987`

```
cap = cv2.VideoCapture(tmp_path)
```

Lee el video y achica cada frame a un ancho máximo de 640 (MAX_FRAME_WIDTH, línea 84).

```
 1244      cap = cv2.VideoCapture(tmp_path)
 1245      if not cap.isOpened():
 1246          raise HTTPException(status_code=400, detail="No se pudo leer
el video enviado.")
 1247
 1248      frames = []
 1249      while True:
 1250          ok, frame = cap.read()
 1251          if not ok:
 1252              break
 1253          # La cámara del navegador graba en Full HD (1920x1080) --
HRNet no
 1254          # necesita esa resolución para encontrar la cara, se achica
ACÁ,
 1255          # antes de meterla al pipeline, preservando el aspecto
(igual que
```

↓

2

Toma muy corta

Código / procedimiento

server/main.py:1013 constante server/main.py:52

```
if total < MIN_FRAMES_VALID
```

Con menos de 15 frames se rechaza la toma.

```
 1272      if total < MIN_FRAMES_VALID:
 1273          return {
 1274              "valid": False,
 1275              "reason": f"toma muy corta ({total} frames, minimo
{MIN_FRAMES_VALID}) "
 1276                        "-- probablemente no se dijo la frase
completa",
 1277          }
 1278
 1279      landmarks_por_frame = detect_landmarks_batch(landmarker, frames)
 1280      print(f"[predict] landmarks detectados ({time.time() -
t_start:.1f}s)")
 1281
 1282      positions = []
 1283      detected = 0
```

```
   78  MIN_FRAMES_VALID = 15           # menos que esto = probablemente no
se dijo la frase completa
   79  MIN_DETECTION_RATE_VALID = 0.6  # menos del 60% de frames con cara
detectada = toma mala
   80
   81  # Ancho máximo de cada frame ANTES de mandarlo a HRNet -- la cámara
del
   82  # navegador graba en Full HD (1920px), muchísimo más de lo necesario
para
   83  # detectar una cara. 640px es lo mismo que usa probar_modelo.py
(FRAME_SIZE).
   84  MAX_FRAME_WIDTH = 640
   85
   86  # Umbral usado SOLO por _run_audio_only_pipeline (modo "solo voz",
sin
   87  # video) -- ahí no hay señal visual para cruzar, así que decide el
audio
   88  # solo con su propio umbral de confianza.
   89  AUDIO_MIN_CONFIDENCE = 0.60
```

↓

3 Cara no detectada

Código / procedimiento

`server/main.py:1027` constante `server/main.py:53`

```
detection_rate < MIN_DETECTION_RATE_VALID
```

Si la cara se detecta en menos del 60% de los frames se rechaza la toma.

```
 1296      if filled is None or detection_rate < MIN_DETECTION_RATE_VALID:
 1297          return {
 1298              "valid": False,
 1299              "reason": f"cara detectada solo en {detected}/{total}
frames "
 1300                        f"({detection_rate*100:.0f}%) -- encuadre
malo, repetí la toma",
 1301          }
 1302
 1303      sequence, mask = preprocess_sequence(filled, max_frames)
 1304      sequence, mask = sequence.to(DEVICE), mask.to(DEVICE)
 1305      with torch.inference_mode():
 1306          logits = model(sequence, src_key_padding_mask=mask)
 1307          probs = torch.softmax(logits, dim=1)[0]
```

```
   79  MIN_DETECTION_RATE_VALID = 0.6  # menos del 60% de frames con cara
detectada = toma mala
   80
   81  # Ancho máximo de cada frame ANTES de mandarlo a HRNet -- la cámara
del
   82  # navegador graba en Full HD (1920px), muchísimo más de lo necesario
para
   83  # detectar una cara. 640px es lo mismo que usa probar_modelo.py
(FRAME_SIZE).
   84  MAX_FRAME_WIDTH = 640
   85
   86  # Umbral usado SOLO por _run_audio_only_pipeline (modo "solo voz",
sin
   87  # video) -- ahí no hay señal visual para cruzar, así que decide el
audio
   88  # solo con su propio umbral de confianza.
   89  AUDIO_MIN_CONFIDENCE = 0.60
   90
```

get_mouth_crop (extraer_landmarks_npy.py:433) existe pero el modelo no lo usa: recibe solo las coordenadas de los 20 puntos, sin píxeles de imagen.

FASE 2B

Extracción de audio

Del WAV al embedding de 1024 números que alimenta al clasificador.

ENTRA	SALE
Audio WAV (cualquier frecuencia)	Vector de 1024 números, normalizado

Modelo: **wav2vec2-xlsr-53-espeak-cv-ft (aud:45)**

Frecuencia: **16000 Hz (aud:46)**

Embedding: **1024 (aud:260)**

Vocabulario: **392 fonemas**

Recorrido

1 load_model Modelo externo ya entrenado

`scripts/audio_pronunciation.py:129` en el servidor `server/main.py:513`

```
def load_model(device=None)
```

Carga wav2vec2 (Wav2Vec2ForCTC, línea 230) con los pesos descargados a mano. Se carga una sola vez al arrancar.

```
  208  def load_model(device=None):
  209      import json
  210      from huggingface_hub import hf_hub_download
  211      from transformers import Wav2Vec2Config,
Wav2Vec2FeatureExtractor, Wav2Vec2ForCTC
  212
  213      device = device or ("cuda" if torch.cuda.is_available() else
"cpu")
  214      print(f"Cargando {MODEL_NAME} (primera vez tarda, baja
~1.2GB)...")
  215
  216      feature_extractor =
Wav2Vec2FeatureExtractor.from_pretrained(MODEL_NAME)
  217
  218      # OJO: a propósito NO se usa
Wav2Vec2ForCTC.from_pretrained(MODEL_NAME)
  219      # -- ese checkpoint solo existe en el repo como
pytorch_model.bin (sin
  220      # versión safetensors), y transformers >=4.5x bloquea
directamente
  221      # cargar .bin con torch.load si torch < 2.6 (CVE-2025-32434),
aunque el
  222      # torch instalado (2.5.1, el que usa el resto del proyecto
para el
```

```
  599          audio_model, audio_fe, audio_vocab, _ =
load_audio_model(device=DEVICE)
  600          _state.update(audio_model=audio_model,
audio_feature_extractor=audio_fe, audio_vocab=audio_vocab)
  601          print("Modelo de audio listo.")
  602      except Exception as e:
  603          print(f"[AVISO] No se pudo cargar el modelo de audio ({e}) -
- "
  604                "el veredicto va a usar solo el video, sin análisis de
sonido.")
  605
  606      # Clasificador de audio entrenado con clips reales -- opcional
(puede no
  607      # existir todavía si nadie corrió train_audio_classifier.py).
Sin esto,
  608      # el análisis de audio sigue funcionando con el GOP genérico de
arriba,
```

```
  609      # solo que sin el refuerzo directo entrenado con tus datos.
  610      if os.path.exists(AUDIO_CLASSIFIER_PATH):
```

↓

2

load_audio_16k

Código / procedimiento

`scripts/audio_pronunciation.py:155`

```
def load_audio_16k(wav_path)
```

Lee el WAV, lo pasa a mono y lo remuestrea a 16 kHz. Así absorbe la frecuencia del navegador.

```
  245  def load_audio_16k(wav_path):
  246      """Devuelve un tensor 1D float32 mono a 16kHz, sea cual sea el
  247      sample rate/canales originales del .wav grabado."""
  248      waveform, sr = torchaudio.load(wav_path)
  249      if waveform.shape[0] > 1:
  250          waveform = waveform.mean(dim=0, keepdim=True)  # a mono
  251      if sr != TARGET_SAMPLE_RATE:
  252          waveform = torchaudio.functional.resample(waveform, sr,
TARGET_SAMPLE_RATE)
  253      return waveform.squeeze(0)
```

↓

3

extract_embedding

Código / procedimiento

`scripts/audio_pronunciation.py:167`

```
def extract_embedding(wav_path, model, feature_extractor, device)
```

Pasa el audio por el encoder de wav2vec2 (línea 278) y promedia en el tiempo (línea 279). Da un vector de 1024.

```
  263  def extract_embedding(wav_path, model, feature_extractor, device):
  264      """Vector fijo (EMBEDDING_DIM,) que resume el AUDIO completo
del clip --
  265      a diferencia de score_pronunciation() (que da un puntaje por
fonema
  266      esperado), esto es lo que usa el clasificador entrenado con
datos reales
  267      (train_audio_classifier.py) para decidir directamente
"correcto" vs
  268      cada tipo de error, igual que el modelo visual hace con
landmarks.
  269
  270      Es el promedio en el tiempo del último estado oculto del
encoder wav2vec2
  271      (antes de la cabeza CTC de fonemas) -- una representación
acústica
  272      general del clip, no atada a ninguna palabra esperada."""
  273      waveform = load_audio_16k(wav_path)
  274      inputs = feature_extractor(waveform.numpy(),
sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
  275      with torch.inference_mode():
  276          # model.wav2vec2 es el encoder base (sin la cabeza de
fonemas) --
  277          # Wav2Vec2ForCTC lo expone como submódulo con ese nombre.
```

↓

4

Normalización

Código / procedimiento

en vivo `server/main.py:902`

entrenamiento `scripts/train_audio_classifier.py:196`

```
emb_norm = (emb - audio_clf_mean) / audio_clf_std
```

Resta la media y divide por el desvío calculados con el set de entrenamiento. Se guardan dentro del modelo.

```
 1106          emb_norm = (emb - _state["audio_clf_mean"]) /
_state["audio_clf_std"]
 1107          # mean/std se guardaron con shape (1, 1024) (ver
train_audio_
 1108          # classifier.py) -- emb_norm ya sale (1, 1024) por el
broadcasting,
 1109          # así que reshape en vez de unsqueeze (que agregaba una
dimensión de
 1110          # más y rompía el softmax/indexado de abajo).
 1111          x = torch.from_numpy(emb_norm.reshape(1,
-1)).float().to(DEVICE)
 1112          with torch.no_grad():
 1113              probs = torch.softmax(clf(x), dim=1)[0]
 1114          # BUG real detectado: acá se calculaba el argmax sobre las
58 clases
 1115          # completas, sin usar "indices_de_la_palabra" para nada --
si pedías
 1116          # "perro" pero el audio se parecía más a "gota", devolvía
 1117          # "gota_correcto" en vez de reconocer que ni siquiera es la
palabra
```

```
  251      mean = X[train_idx].mean(axis=0, keepdims=True)
  252      std = X[train_idx].std(axis=0, keepdims=True) + 1e-6
  253
  254      def to_tensor(idx):
  255          xs = (X[idx] - mean) / std
  256          return torch.from_numpy(xs).float().to(device),
torch.from_numpy(y[idx]).long().to(device)
  257
  258      x_train, y_train = to_tensor(train_idx)
  259      x_val, y_val = to_tensor(val_idx)
  260
  261      head = AudioClassifierHead(EMBEDDING_DIM, HIDDEN_DIM,
len(class_names), DROPOUT).to(device)
  262      optimizer = torch.optim.Adam(head.parameters(), lr=LR,
weight_decay=WEIGHT_DECAY)
```

Solo entrenamiento

1

find_labeled_clips Código / procedimiento

`scripts/train_audio_classifier.py:29`

```
def find_labeled_clips()
```

Junta los clips por clase. Si existe la caché de embeddings, manda la caché.

```
   59  def find_labeled_clips():
   60      """{clase: [ruta.wav o ruta.npy, ...]} -- el cache de
embeddings
   61      (data/audio_embeddings_cache/, formato "<clase>_NNNN.npy") es
la fuente
   62      de verdad: ahí está TODO el dataset real ya extraído (miles de
clips),
   63      mientras que data/sesiones_continuas/ puede tener carpetas
viejas o
   64      incompletas (clips sueltos que quedaron sin archivar, carpetas
de
   65      pruebas de otras épocas del proyecto como "puerco") --
entrenar desde
   66      ahí sin querer se comería solo esas sobras en vez del dataset
completo.
   67      Por eso el cache manda siempre que exista y tenga algo;
sesiones_continuas
   68      solo se usa como fallback si el cache no existe todavía."""
   69      if os.path.isdir(CACHE_DIR):
   70          por_clase = {}
   71          for f in sorted(os.listdir(CACHE_DIR)):
   72              if not f.endswith(".npy"):
   73                  continue
   74              clase = re.sub(r"_\d+$", "", os.path.splitext(f)[0])
```

↓

2 build_embeddings

Código / procedimiento

`scripts/train_audio_classifier.py:55`

caché `scripts/train_audio_classifier.py:69 a 119`

```
def build_embeddings(clips_por_clase, model, feature_extractor, device)
```

Calcula el embedding de cada clip y lo guarda en data/audio_embeddings_cache/ para no recalcularlo.

```
   94  def build_embeddings(clips_por_clase, model, feature_extractor,
device):
   95      """Extrae (o lee de la caché) el embedding de cada clip.
Devuelve
   96      (X, y, class_names) -- X: (N, 1024) float32, y: (N,) int,
class_names:
   97      lista ordenada usada para mapear y -> nombre de clase.
   98
   99      Los items de clips_por_clase pueden ser rutas a .wav (caso
normal) o
  100      directo a un .npy ya cacheado (caso fallback sin
sesiones_continuas,
  101      ver find_labeled_clips) -- en ese caso se carga tal cual, sin
necesitar
  102      el modelo de wav2vec2."""
  103      os.makedirs(CACHE_DIR, exist_ok=True)
  104      class_names = sorted(clips_por_clase.keys())
  105      X, y = [], []
  106      total = sum(len(v) for v in clips_por_clase.values())
  107      done = 0
  108      for clase in class_names:
  109          for item_path in clips_por_clase[clase]:
  110              if item_path.lower().endswith(".npy"):
  111                  emb = np.load(item_path)
```

```
  116                      emb = np.load(cache_path)
  117                  else:
  118                      emb = extract_embedding(item_path, model,
feature_extractor, device)
  119                      np.save(cache_path, emb)
  120              X.append(emb)
  121              y.append(class_names.index(clase))
  122              done += 1
  123              if done % 20 == 0 or done == total:
  124                  print(f"  embeddings: {done}/{total}")
  125      return np.stack(X).astype(np.float32), np.array(y,
dtype=np.int64), class_names
  126
  127
```

FASE 3A

Modelo de video: LipReadingConformer

TCN + Conformer sobre la secuencia de landmarks. Es tu modelo entrenado.

ENTRA	SALE
Tensor (1, 60, 120) y máscara (1, 60)	Logits (1, 66) y luego softmax

input_dim: **120** · max_frames: **60** · Clases: **66** · hidden_dim: **128** · Dropout: **0.3**

Épocas: **50** · Batch: **32** · lr: **1e-3** · Weight decay: **1e-3** · Label smoothing: **0.1**

Calentamiento: **5 épocas** · Paciencia: **15** · Validación: **20% estratificado**

Acierto exacto: **70.27%** · Bien o mal: **78.16%** · Confianza suficiente: **87.75%**

Latencia: **15.38 ms**

Capas, en el orden en que pasa el dato

Entrada `(B, 60, 120)` `scripts/train_landmarks_transformer.py:213`

↓

input_proj Linear 408 a 128 `(B, 60, 128)` `scripts/train_landmarks_transformer.py:184`

↓

TCN: 3 bloques, dilatación 1, 2, 4, kernel 3 `(B, 60, 128)`

`scripts/train_landmarks_transformer.py:186`

↓

PositionalEncoding `(B, 60, 128)` `scripts/train_landmarks_transformer.py:190`

↓

Conformer: 2 capas, 8 cabezas, feed-forward 256, kernel 15 `(B, 60, 128)`

`scripts/train_landmarks_transformer.py:192`

↓

masked_mean_pool (ignora el relleno) `(B, 128)`

`scripts/train_landmarks_transformer.py:217`

↓

classifier 128 a 64 a 66	(B, 66)	scripts/train_landmarks_transformer.py:198

Piezas de la arquitectura (train_landmarks_transformer.py, copia en server/main.py)

1 LipReadingConformer

Tu modelo entrenado

entrenamiento `scripts/train_landmarks_transformer.py:179`

copia en el servidor `server/main.py:387`

```
class LipReadingConformer(num_classes, input_dim, hidden_dim=128,
n_layers=2, n_heads=8, ...)
```

El modelo completo. forward en la línea 292 y forward_features en la 278.

```
  244  class LipReadingConformer(nn.Module):
  245      # Defaults pensados para dataset CHICO (pocas clases / pocas
muestras por
  246      # clase, como ahora mismo con 3-19 de las 57 palabras). Con
~4M de
  247      # parámetros el Conformer memorizaba el train set en vez de
generalizar
  248      # (Train Loss bajaba, Val Loss no). Cuando tengas el dataset
completo
  249      # (57 clases, ~500 clips c/u) subí hidden_dim=256, n_layers=4,
  250      # dim_feedforward=512 -- ahí sí va a hacer falta esa
capacidad.
  251      def __init__(self, num_classes, input_dim, hidden_dim=128,
  252                   n_layers=2, n_heads=8, dim_feedforward=256,
  253                   conv_kernel_size=15, tcn_blocks=3,
tcn_kernel_size=3, dropout=0.3):
  254          super().__init__()
  255          self.input_proj = nn.Linear(input_dim, hidden_dim)
  256
  257          # Extracción de features locales (TCN) ANTES de pasarle la
secuencia
  258          # al Conformer -- ver TCNFeatureExtractor más arriba.
  259          self.tcn = TCNFeatureExtractor(
  260              hidden_dim, n_blocks=tcn_blocks,
```

```
  465  class LipReadingConformer(nn.Module):
  466      def __init__(self, num_classes, input_dim, hidden_dim=128,
  467                   n_layers=2, n_heads=8, dim_feedforward=256,
  468                   conv_kernel_size=15, tcn_blocks=3,
tcn_kernel_size=3, dropout=0.3):
  469          super().__init__()
  470          self.input_proj = nn.Linear(input_dim, hidden_dim)
  471          self.tcn = TCNFeatureExtractor(hidden_dim,
n_blocks=tcn_blocks, kernel_size=tcn_kernel_size, dropout=dropout)
  472          self.pos_encoder = PositionalEncoding(hidden_dim)
  473          self.conformer_encoder = ConformerEncoder(
  474              d_model=hidden_dim, n_layers=n_layers,
n_heads=n_heads,
  475              dim_feedforward=dim_feedforward,
conv_kernel_size=conv_kernel_size, dropout=dropout,
```

```
  476          )
  477          self.classifier = nn.Sequential(
  478              nn.Linear(hidden_dim, 64), nn.ReLU(),
nn.Dropout(dropout), nn.Linear(64, num_classes),
  479          )
  480
  481      def forward(self, x, src_key_padding_mask=None):
  482          x = self.input_proj(x)
```

↓

2 TCNFeatureExtractor / TCNBlock

Tu modelo entrenado

TCNFeatureExtractor `scripts/train_landmarks_transformer.py:65`

TCNBlock `scripts/train_landmarks_transformer.py:43`

servidor `server/main.py:282 y 338`

```
TCNBlock(channels, kernel_size=3, dilation=1, dropout=0.1)
```

Convoluciones 1D con dilatación creciente (1, 2, 4), BatchNorm y conexión residual. Captan el movimiento local.

```
  105  class TCNFeatureExtractor(nn.Module):
  106      def __init__(self, channels, n_blocks=3, kernel_size=3,
dropout=0.1):
  107          super().__init__()
  108          self.blocks = nn.ModuleList([
  109              TCNBlock(channels, kernel_size=kernel_size, dilation=2
** i, dropout=dropout)
  110              for i in range(n_blocks)
  111          ])
  112
  113      def forward(self, x, key_padding_mask=None):
  114          for block in self.blocks:
  115              x = block(x, key_padding_mask=key_padding_mask)
  116          return x
```

```
   82  class TCNBlock(nn.Module):
   83      def __init__(self, channels, kernel_size=3, dilation=1,
dropout=0.1):
   84          super().__init__()
   85          padding = (kernel_size - 1) * dilation // 2  # padding
simétrico -- no causal, mantiene largo T
   86          self.conv1 = nn.Conv1d(channels, channels, kernel_size,
padding=padding, dilation=dilation)
   87          self.bn1 = nn.BatchNorm1d(channels)
   88          self.conv2 = nn.Conv1d(channels, channels, kernel_size,
padding=padding, dilation=dilation)
   89          self.bn2 = nn.BatchNorm1d(channels)
   90          self.relu = nn.ReLU()
   91          self.dropout = nn.Dropout(dropout)
   92
   93      def forward(self, x, key_padding_mask=None):
   94          # x: (B, T, C)
   95          residual = x
   96          x = x.transpose(1, 2)  # (B, C, T) -- Conv1d espera el
tiempo al final
   97          if key_padding_mask is not None:
   98              x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
   99          x = self.dropout(self.relu(self.bn1(self.conv1(x))))
```

```
        x = self.bn2(self.conv2(x))
```

```
class TCNFeatureExtractor(nn.Module):
    def __init__(self, channels, n_blocks=3, kernel_size=3, dropout=0.1):
        super().__init__()
        self.blocks = nn.ModuleList([
            TCNBlock(channels, kernel_size=kernel_size, dilation=2 ** i, dropout=dropout)
            for i in range(n_blocks)
        ])

    def forward(self, x, key_padding_mask=None):
        for block in self.blocks:
            x = block(x, key_padding_mask=key_padding_mask)
        return x
```

```
class TCNBlock(nn.Module):
    def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.1):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn2 = nn.BatchNorm1d(channels)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.dropout(self.relu(self.bn1(self.conv1(x))))
        x = self.bn2(self.conv2(x))
        x = x.transpose(1, 2)
        return self.relu(residual + self.dropout(x))
```

↓

3

PositionalEncoding

Tu modelo entrenado

`scripts/train_landmarks_transformer.py:29` servidor `server/main.py:250`

```
class PositionalEncoding(d_model, max_len=500)
```

Suma una señal sinusoidal para que el modelo sepa el orden de los frames.

```
  62  class PositionalEncoding(nn.Module):
  63      def __init__(self, d_model, max_len=500):
  64          super().__init__()
  65          pe = torch.zeros(max_len, d_model)
  66          position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
  67          div_term = torch.exp(torch.arange(0, d_model, 2).float() *
(-math.log(10000.0) / d_model))
  68          pe[:, 0::2] = torch.sin(position * div_term)
  69          pe[:, 1::2] = torch.cos(position * div_term)
  70          self.register_buffer("pe", pe.unsqueeze(0))
  71
  72      def forward(self, x):
  73          return x + self.pe[:, : x.size(1)]
```

```
 324  class PositionalEncoding(nn.Module):
 325      def __init__(self, d_model, max_len=500):
 326          super().__init__()
 327          pe = torch.zeros(max_len, d_model)
 328          position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
 329          div_term = torch.exp(torch.arange(0, d_model, 2).float() *
(-math.log(10000.0) / d_model))
 330          pe[:, 0::2] = torch.sin(position * div_term)
 331          pe[:, 1::2] = torch.cos(position * div_term)
 332          self.register_buffer("pe", pe.unsqueeze(0))
 333
 334      def forward(self, x):
 335          return x + self.pe[:, : x.size(1)]
```

↓

4 ConformerEncoder / ConformerBlock

Tu modelo entrenado

Encoder `scripts/train_landmarks_transformer.py:164`

Block `scripts/train_landmarks_transformer.py:131`

servidor `server/main.py:364 y 420`

```
ConformerBlock(d_model, n_heads, dim_feedforward, conv_kernel_size=15,
dropout=0.1)
```

Cada bloque: feed-forward a medio peso, atención, módulo convolucional y feed-forward a medio peso.

```
  224  class ConformerEncoder(nn.Module):
  225      def __init__(self, d_model, n_layers, n_heads, dim_feedforward,
  226                   conv_kernel_size=15, dropout=0.1):
  227          super().__init__()
  228          self.layers = nn.ModuleList([
  229              ConformerBlock(d_model, n_heads, dim_feedforward,
conv_kernel_size, dropout)
  230              for _ in range(n_layers)
  231          ])
  232
  233      def forward(self, x, src_key_padding_mask=None):
  234          for layer in self.layers:
  235              x = layer(x, src_key_padding_mask=src_key_padding_mask)
  236          return x
```

```
  187  class ConformerBlock(nn.Module):
  188      def __init__(self, d_model, n_heads, dim_feedforward,
conv_kernel_size=15, dropout=0.1):
  189          super().__init__()
  190          self.ff1 = ConformerFeedForward(d_model, dim_feedforward,
dropout)
  191
  192          self.self_attn_norm = nn.LayerNorm(d_model)
  193          self.self_attn = nn.MultiheadAttention(
  194              d_model, n_heads, dropout=dropout, batch_first=True
  195          )
  196          self.attn_dropout = nn.Dropout(dropout)
  197
  198          self.conv_module = ConformerConvModule(d_model,
conv_kernel_size, dropout)
  199
  200          self.ff2 = ConformerFeedForward(d_model, dim_feedforward,
dropout)
  201          self.final_norm = nn.LayerNorm(d_model)
  202
  203      def forward(self, x, src_key_padding_mask=None):
  204          # 1. FFN macaron (mitad de peso)
```

```
        x = x + 0.5 * self.ff1(x)

```

```
class ConformerEncoder(nn.Module):
    def __init__(self, d_model, n_layers, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            ConformerBlock(d_model, n_heads, dim_feedforward, conv_kernel_size, dropout)
            for _ in range(n_layers)
        ])

    def forward(self, x, src_key_padding_mask=None):
        for layer in self.layers:
            x = layer(x, src_key_padding_mask=src_key_padding_mask)
        return x
```

```
class ConformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.ff1 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.self_attn_norm = nn.LayerNorm(d_model)
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.attn_dropout = nn.Dropout(dropout)
        self.conv_module = ConformerConvModule(d_model, conv_kernel_size, dropout)
        self.ff2 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.final_norm = nn.LayerNorm(d_model)

    def forward(self, x, src_key_padding_mask=None):
        x = x + 0.5 * self.ff1(x)
        residual = x
        x_norm = self.self_attn_norm(x)
        attn_out, _ = self.self_attn(x_norm, x_norm, x_norm, key_padding_mask=src_key_padding_mask, need_weights=False)
        x = residual + self.attn_dropout(attn_out)
        x = self.conv_module(x,
```

↓

5 ConformerFeedForward / ConformerConvModule

Tu modelo entrenado

FeedForward `scripts/train_landmarks_transformer.py:79`

ConvModule `scripts/train_landmarks_transformer.py:96`

servidor `server/main.py:296 y 390`

```
ConformerConvModule: pointwise, GLU, depthwise (kernel 15), BatchNorm,
SiLU, pointwise
```

El módulo convolucional es lo que distingue al Conformer de un Transformer común.

```
  122  class ConformerFeedForward(nn.Module):
  123      """Feed-forward "macaron" -- se usa DOS veces por bloque, cada
una con
  124      la mitad de peso (residual * 0.5), a diferencia de un
Transformer
  125      normal que tiene una sola FFN de peso completo."""
  126
  127      def __init__(self, d_model, dim_feedforward, dropout):
  128          super().__init__()
  129          self.net = nn.Sequential(
  130              nn.LayerNorm(d_model),
  131              nn.Linear(d_model, dim_feedforward),
  132              nn.SiLU(),  # Swish -- la activación estándar de
Conformer
  133              nn.Dropout(dropout),
  134              nn.Linear(dim_feedforward, d_model),
  135              nn.Dropout(dropout),
  136          )
  137
  138      def forward(self, x):
  139          return self.net(x)
```

```
  142  class ConformerConvModule(nn.Module):
  143      """El módulo que distingue a Conformer de un Transformer
plano: una
  144      convolución depthwise a lo largo del tiempo, que captura
patrones
  145      LOCALES (ej. la transición rápida entre dos formas de boca)
que la
  146      self-attention por sí sola modela peor.
  147
  148      Pointwise conv -> GLU -> Depthwise conv -> BatchNorm -> Swish
-> Pointwise conv
  149      """
  150
```

```
    def __init__(self, d_model, kernel_size=15, dropout=0.1):
        super().__init__()
        self.layer_norm = nn.LayerNorm(d_model)
        self.pointwise_conv1 = nn.Conv1d(d_model, 2 * d_model,
kernel_size=1)
        self.glu = nn.GLU(dim=1)
        padding = (kernel_size - 1) // 2
        self.depthwise_conv = nn.Conv1d(
            d_model, d_model, kernel_size=kernel_size,
padding=padding, groups=d_model
```

```
class ConformerFeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, dim_feedforward),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(dim_feedforward, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)
```

```
class ConformerConvModule(nn.Module):
    def __init__(self, d_model, kernel_size=15, dropout=0.1):
        super().__init__()
        self.layer_norm = nn.LayerNorm(d_model)
        self.pointwise_conv1 = nn.Conv1d(d_model, 2 * d_model,
kernel_size=1)
        self.glu = nn.GLU(dim=1)
        padding = (kernel_size - 1) // 2
        self.depthwise_conv = nn.Conv1d(d_model, d_model,
kernel_size=kernel_size, padding=padding, groups=d_model)
        self.batch_norm = nn.BatchNorm1d(d_model)
        self.swish = nn.SiLU()
        self.pointwise_conv2 = nn.Conv1d(d_model, d_model,
kernel_size=1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = self.layer_norm(x)
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.pointwise_conv1(x)
        x = self.glu(x)
```

↓

6 masked_mean_pool

Tu modelo entrenado

`scripts/train_landmarks_transformer.py:217` servidor `server/main.py:378`

```
def masked_mean_pool(x, src_key_padding_mask)
```

Promedia en el tiempo ignorando los frames de relleno.

```
  298  def masked_mean_pool(x, src_key_padding_mask):
  299      """Global Average Pooling que IGNORA los frames de padding.
  300
  301      x.mean(dim=1) directo diluye la representación con los frames en
cero
  302      del padding -- acá se promedia solo sobre los frames reales,
usando la
  303      máscara invertida (src_key_padding_mask es True en el padding).
  304      """
  305      if src_key_padding_mask is None:
  306          return x.mean(dim=1)
  307
  308      real_mask = (~src_key_padding_mask).unsqueeze(-1).float()   #
(B, T, 1), 1=real, 0=padding
  309      summed = (x * real_mask).sum(dim=1)                          #
(B, hidden_dim)
  310      counts = real_mask.sum(dim=1).clamp(min=1.0)                 #
(B, 1) -- evita /0 si T real=0
  311      return summed / counts
```

```
  456  def masked_mean_pool(x, src_key_padding_mask):
  457      if src_key_padding_mask is None:
  458          return x.mean(dim=1)
  459      real_mask = (~src_key_padding_mask).unsqueeze(-1).float()
  460      summed = (x * real_mask).sum(dim=1)
  461      counts = real_mask.sum(dim=1).clamp(min=1.0)
  462      return summed / counts
```

↓

7

make_padding_mask

Código / procedimiento

`scripts/train_landmarks_transformer.py:227` servidor `server/main.py:412`

```
def make_padding_mask(lengths, max_len, device)
```

Marca con True los frames de relleno.

```
  314  def make_padding_mask(lengths, max_len, device):
  315      """True donde hay padding (para que el encoder -- y el masked
pooling --
  316      lo ignoren)."""
  317      idx = torch.arange(max_len, device=device).unsqueeze(0)
  318      return idx >= lengths.unsqueeze(1)
```

```
  490  def make_padding_mask(lengths, max_len, device):
  491      idx = torch.arange(max_len, device=device).unsqueeze(0)
  492      return idx >= lengths.unsqueeze(1)
```

Entrenamiento

1

build_dataloaders

Código / procedimiento

`scripts/lip_reading_dataset.py:99`

```
def build_dataloaders(data_dir, max_frames=60, batch_size=32,
val_ratio=0.2, seed=42, ...)
```

Arma los datos de train y validación separando por clase (estratificado).

```
  153  def build_dataloaders(data_dir=DEFAULT_DATA_DIR, max_frames=60,
batch_size=32,
  154                        val_ratio=0.2, seed=42, num_workers=0,
augment_train=True):
  155      """Arma el dataset completo, lo separa en train/val a nivel de
ARCHIVOS
  156      (no de tensores ya cargados) para poder darle augmentation
SOLO al
  157      split de entrenamiento (nunca a val), y devuelve los
DataLoaders listos
  158      junto con un dataset "de referencia" (para leer
class_names/landmark_dim).
  159
  160      Split ESTRATIFICADO por clase: separa val_ratio de CADA clase
por
  161      separado (no un permutation al azar sobre TODO el dataset) --
con
  162      clases de tamaños muy distintos (ej. "perro_correcto" con
cientos de
  163      clips vs "perro_incorrecto_dentalizacion" con 50), un split
global podía
  164      dejar alguna clase chica con pocas o CERO muestras de
validación."""
  165      reference = LipReadingDataset(data_dir=data_dir,
```

↓

2 LipReadingDataset Código / procedimiento

clase scripts/lip_reading_dataset.py:20

__getitem__ scripts/lip_reading_dataset.py:73

```
class LipReadingDataset(Dataset)
```

Lee los .npy y el label_map.json. Rellena o recorta cada secuencia a 60 frames.

```
   46  class LipReadingDataset(Dataset):
   47      """Lee .npy de landmarks y arma (secuencia, label,
longitud_real).
   48
   49      Args:
   50          data_dir: carpeta con subcarpetas por palabra
(label_map.json adentro).
   51          max_frames: longitud fija de secuencia (padding con ceros
o truncado).
   52          file_paths: lista explícita de archivos a usar (para
separar train/val
   53              sin compartir estado de augmentation entre ambos
splits). Si es
   54              None, usa TODOS los .npy de data_dir.
   55          augment: si True, aplica aumento de datos en cada
__getitem__
   56              (ruido gaussiano, shift/scale, time masking). Usar
SOLO en el
   57              split de entrenamiento -- nunca en validación/test.
   58      """
   59
   60      def __init__(self, data_dir=DEFAULT_DATA_DIR, max_frames=60,
file_paths=None, augment=False):
   61          self.data_dir = data_dir
```

```
  127      def __getitem__(self, idx):
  128          path = self.file_paths[idx]
  129          sequence = np.load(path).astype(np.float32)  # (T,
landmark_dim)
  130          label = self._label_from_path(path)
  131
  132          if self.augment:
  133              sequence = self._apply_augmentation(sequence)
  134
  135          T = sequence.shape[0]
  136          if T < self.max_frames:
  137              pad = np.zeros((self.max_frames - T,
sequence.shape[1]), dtype=np.float32)
  138              sequence = np.concatenate([sequence, pad], axis=0)
  139              real_length = T
```

```
  140          elif T > self.max_frames:
  141              sequence = sequence[: self.max_frames]
  142              real_length = self.max_frames
  143          else:
  144              real_length = T
  145
  146          return (
  147              torch.from_numpy(sequence),                    #
(max_frames, landmark_dim)
```

↓

3 _apply_augmentation

Código / procedimiento

`scripts/lip_reading_dataset.py:58`

```
def _apply_augmentation(self, sequence)
```

Solo en train: ruido 0.003 (línea 107), desplazamiento y escala (112 a 114), enmascarado de 8 a 12% de frames (119 a 123).

```
   99      def _apply_augmentation(self, sequence):
  100          """Aumento de datos SOLO para entrenamiento -- opera sobre
la
  101          secuencia (T, landmark_dim) ANTES de hacer padding, para
no meter
  102          ruido en los frames de relleno."""
  103          # Ruido gaussiano leve (simula jitter de detección de
landmarks).
  104          # Valor chico a propósito: la señal normalizada ya es de
por sí
  105          # pequeña (std temporal típica ~0.02-0.03), así que un
ruido más
  106          # fuerte que eso tapa el movimiento real en vez de
robustecer.
  107          sequence = sequence + np.random.normal(0, 0.003,
sequence.shape).astype(np.float32)
  108
  109          # Shift/scale aleatorio leve de las coordenadas (variación
residual
  110          # de encuadre/escala que la normalización geométrica no
cubre del
  111          # todo) -- valores chicos a propósito, para no destruir la
señal.
  112          shift = np.random.uniform(-0.01, 0.01, size=(1,
```

↓

4

train

Código / procedimiento

`scripts/train_landmarks_transformer.py:407`

```
def train(epochs=50, batch_size=32, max_frames=60, lr=1e-3,
weight_decay=1e-3, ...)
```

Pesos por clase (570 a 574), pérdida (583), AdamW (584), scheduler (585) y bucle de épocas (594).

```
  546  def train(epochs=50, batch_size=32, max_frames=60, lr=1e-3,
weight_decay=1e-3,
  547            val_ratio=0.2, early_stopping_patience=15,
warmup_epochs=5,
  548            label_smoothing=0.1):
  549      device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
  550      print(f"Entrenando en: {device}")
  551
  552      dataset, train_loader, val_loader = build_dataloaders(
  553          data_dir=RUTA_DATASET, max_frames=max_frames,
batch_size=batch_size,
  554          val_ratio=val_ratio, seed=SEED, augment_train=True,
  555      )
  556      num_classes = len(dataset.class_names)
  557      input_dim = dataset.landmark_dim  # detectado del .npy real,
no hardcodeado
  558      print(f"Clases ({num_classes}): {dataset.class_names}")
  559      print(f"Dimensión de entrada detectada: {input_dim}")
  560      print(f"Total muestras: {len(dataset)} | Train:
{len(train_loader.dataset)} | "
  561            f"Val: {len(val_loader.dataset)}")
  562
  563      model = LipReadingConformer(num_classes=num_classes,
```

↓

5

run_epoch

Código / procedimiento

`scripts/train_landmarks_transformer.py:272`

```
def run_epoch(model, loader, criterion, optimizer, device, train)
```

Una pasada completa por los datos, entrenando o validando.

```
  370  def run_epoch(model, loader, criterion, optimizer, device, train):
  371      model.train() if train else model.eval()
  372      total_loss, correct, total = 0.0, 0, 0
  373
  374      context = torch.enable_grad() if train else torch.no_grad()
  375      with context:
  376          for sequences, labels, lengths in loader:
  377              sequences = sequences.to(device)
  378              labels = labels.to(device)
  379              lengths = lengths.to(device)
  380              mask = make_padding_mask(lengths, sequences.shape[1],
device)
  381
  382              if train:
  383                  optimizer.zero_grad()
  384
  385              outputs = model(sequences, src_key_padding_mask=mask)
  386              loss = criterion(outputs, labels)
  387
  388              if train:
  389                  loss.backward()
  390                  # Gradient clipping -- los Transformer/Conformer
entrenados desde
  391                  # cero son propensos a picos de gradiente que
```

↓

6 build_warmup_cosine_scheduler / EarlyStopping

Código / procedimiento

scheduler `scripts/train_landmarks_transformer.py:305`

EarlyStopping `scripts/train_landmarks_transformer.py:232`

```
build_warmup_cosine_scheduler(optimizer, epochs, warmup_epochs=5)
```

Calentamiento de 5 épocas y luego coseno. EarlyStopping corta si val_loss no mejora en 15 épocas.

```
  407  def build_warmup_cosine_scheduler(optimizer, epochs,
warmup_epochs=5):
  408      """LR sube linealmente desde ~0 hasta el valor pedido durante
los
  409      primeros `warmup_epochs`, y después decae en coseno.
  410
  411      Sin warmup, un Transformer/Conformer arrancado desde cero con
AdamW y
  412      LR alto (1e-3) suele desestabilizarse en las primeras épocas -
- gradientes
  413      grandes contra pesos todavía aleatorios -- lo que se vio como
Val Loss
  414      "rebotando" en vez de bajar. Empezar suave evita ese quilombo
inicial."""
  415      warmup_epochs = max(1, min(warmup_epochs, epochs - 1)) if
epochs > 1 else 1
  416
  417      def lr_lambda(epoch):
  418          if epoch < warmup_epochs:
  419              return (epoch + 1) / warmup_epochs
  420          progress = (epoch - warmup_epochs) / max(1, epochs -
warmup_epochs)
  421          return 0.5 * (1 + math.cos(math.pi * progress))
  422
```

```
  324  class EarlyStopping:
  325      def __init__(self, patience=8, min_delta=0.001):
  326          self.patience = patience
  327          self.min_delta = min_delta
  328          self.counter = 0
  329          self.best_loss = None
  330          self.early_stop = False
  331
  332      def __call__(self, val_loss):
  333          if self.best_loss is None or val_loss < self.best_loss -
self.min_delta:
  334              self.best_loss = val_loss
  335              self.counter = 0
```

```
336          else:
337              self.counter += 1
338              if self.counter >= self.patience:
339                  self.early_stop = True
```

↓

7

Guardado del modelo

Código / procedimiento

`scripts/train_landmarks_transformer.py:462`

`torch.save({model_state_dict, class_names, max_frames, input_dim, ...})`

Guarda models/best.pth cada vez que mejora val_loss.

```
610              torch.save({
611                  "model_state_dict": model.state_dict(),
612                  "class_names": dataset.class_names,
613                  "max_frames": max_frames,
614                  "input_dim": input_dim,
615                  "val_loss": val_loss,
616                  "val_acc": val_acc,
617              }, MODEL_PATH)
618              print(f"  -> Nuevo mejor modelo guardado (Val Loss:
{val_loss:.4f}, Val Acc: {val_acc:.2f}%)")
619
620          early_stopping(val_loss)
621          if early_stopping.early_stop:
```

↓

8

evaluate_final_report

Código / procedimiento

`scripts/train_landmarks_transformer.py:321`

```
def evaluate_final_report(model, val_loader, class_names, device)
```

Calcula acierto exacto, acierto bien o mal, matriz de confusión y latencia. Escribe outputs/reporte_resultados.md.

```
  440  def evaluate_final_report(model, val_loader, class_names, device):
  441      model.eval()
  442      all_preds, all_labels, all_top3_hit, all_top_probs = [], [],
[], []
  443      latencies_ms = []
  444
  445      with torch.no_grad():
  446          for sequences, labels, lengths in val_loader:
  447              sequences = sequences.to(device)
  448              lengths_dev = lengths.to(device)
  449              mask = make_padding_mask(lengths_dev,
sequences.shape[1], device)
  450
  451              outputs = model(sequences, src_key_padding_mask=mask)
  452              probs = torch.softmax(outputs, dim=1)
  453
  454              top1 = probs.argmax(dim=1).cpu()
  455              top3 = probs.topk(min(3, probs.shape[1]),
dim=1).indices.cpu()
  456              top_prob = probs.max(dim=1).values.cpu()
  457
  458              all_preds.extend(top1.tolist())
  459              all_labels.extend(labels.tolist())
  460              all_top_probs.extend(top_prob.tolist())
```

En el servidor

1

Carga

Código / procedimiento

`server/main.py:454 y 569` `lectura server/main.py:417`

```
model = LipReadingConformer(...); model.load_state_dict(...)
```

safe_torch_load lee data/modelos_cara_completa/best_cara_completa.pth y load_everything arma el modelo de cara completa al arrancar (el de models/best.pth, el de labios, quedó comentado).

```
  568      model = LipReadingConformer(num_classes=len(class_names),
input_dim=input_dim).to(DEVICE)
  569      model.load_state_dict(checkpoint["model_state_dict"])
  570      model.eval()
  571
  572      print("Cargando detector de landmarks (HRNet/FAN)...")
  573      landmarker = build_hrnet_detector()
  574
  575      # Precalentamiento: la PRIMERA vez que HRNet corre, PyTorch
intenta
  576      # compilar la red (torch.compile/dynamo), falla en Windows y cae
a
  577      # modo eager -- ese intento fallido es lo que hace que la
primera
  578      # predicción real tarde ~50-90s en vez de ~10-20s. Absorbemos
ese
  579      # costo ACÁ, antes de aceptar requests, para que la primera
```

```
  569      model.load_state_dict(checkpoint["model_state_dict"])
  570      model.eval()
  571
  572      print("Cargando detector de landmarks (HRNet/FAN)...")
  573      landmarker = build_hrnet_detector()
  574
  575      # Precalentamiento: la PRIMERA vez que HRNet corre, PyTorch
intenta
  576      # compilar la red (torch.compile/dynamo), falla en Windows y cae
a
  577      # modo eager -- ese intento fallido es lo que hace que la
primera
  578      # predicción real tarde ~50-90s en vez de ~10-20s. Absorbemos
ese
  579      # costo ACÁ, antes de aceptar requests, para que la primera
  580      # predicción de un usuario real ya sea rápida.
```

```
  495  def safe_torch_load(path):
  496      try:
  497          return torch.load(path, weights_only=True)
```

```
 498      except Exception:
 499          return torch.load(path, weights_only=False)
```

↓

2 Llamada

Código / procedimiento

`server/main.py:1035` softmax `server/main.py:1035`

```
logits = model(sequence, src_key_padding_mask=mask)
```

Dentro de _run_prediction_pipeline. El softmax da la clase ganadora y su confianza.

```
 1306          logits = model(sequence, src_key_padding_mask=mask)
 1307          probs = torch.softmax(logits, dim=1)[0]
 1308      print(f"[predict] modelo Conformer listo ({time.time() -
t_start:.1f}s total)")
 1309
 1310      sorted_probs, sorted_idx = torch.sort(probs, descending=True)
 1311      top_idx = int(sorted_idx[0].item())
 1312      top_prob = float(sorted_probs[0].item())
 1313      second_prob = float(sorted_probs[1].item()) if len(sorted_probs)
> 1 else 0.0
 1314      margin = top_prob - second_prob
 1315      clase_predicha = class_names[top_idx]
 1316      print(f"[predict] video: clase={clase_predicha} prob=
{top_prob:.2f} margin={margin:.2f} "
 1317            f"(2da opcion: {class_names[int(sorted_idx[1].item())] if
len(sorted_idx) > 1 else '?'} "
```

```
 1307          probs = torch.softmax(logits, dim=1)[0]
 1308      print(f"[predict] modelo Conformer listo ({time.time() -
t_start:.1f}s total)")
 1309
 1310      sorted_probs, sorted_idx = torch.sort(probs, descending=True)
 1311      top_idx = int(sorted_idx[0].item())
 1312      top_prob = float(sorted_probs[0].item())
 1313      second_prob = float(sorted_probs[1].item()) if len(sorted_probs)
> 1 else 0.0
 1314      margin = top_prob - second_prob
 1315      clase_predicha = class_names[top_idx]
 1316      print(f"[predict] video: clase={clase_predicha} prob=
{top_prob:.2f} margin={margin:.2f} "
 1317            f"(2da opcion: {class_names[int(sorted_idx[1].item())] if
len(sorted_idx) > 1 else '?'} "
 1318            f"{second_prob:.2f}) | esperado={palabra_esperada}")
```

3 _parse_clase_predicha

Código / procedimiento

server/main.py:63 uso server/main.py:1047

```
def _parse_clase_predicha(nombre_clase)
```

Convierte una clase como perro_correcto en (palabra, correcta, tipo de error).

```
  115  def _parse_clase_predicha(nombre_clase):
  116      """'perro_correcto' -> ('perro', True, None).
  117      'perro_incorrecto' -> ('perro', False, None).
  118      'perro_incorrecto_lambdacismo' -> ('perro', False,
'lambdacismo').
  119      Si la clase no tiene ninguno de los dos marcadores (checkpoint
viejo o
  120      mal entrenado), devuelve (nombre_clase, None, None) -- se trata
como "no
  121      se pudo determinar corrección" en vez de asumir un valor."""
  122      if nombre_clase.endswith(SUFIJO_CORRECTO):
  123          return nombre_clase[: -len(SUFIJO_CORRECTO)], True, None
  124
  125      idx = nombre_clase.find(MARCADOR_INCORRECTO)
  126      if idx != -1:
  127          palabra = nombre_clase[:idx]
  128          resto = nombre_clase[idx + len(MARCADOR_INCORRECTO):]
  129          subtipo = resto[1:] if resto.startswith("_") else None
  130          return palabra, False, subtipo or None
  131
  132      return nombre_clase, None, None
```

```
 1319      palabra, pronunciacion_correcta, tipo_error =
_parse_clase_predicha(clase_predicha)
 1320
 1321      # Cada paso del camino (sonido suelto / doble / a medias /
completa) es
 1322      # una FAMILIA de clases distinta ("perro", "perro_doble",
 1323      # "perro_medias", ...) -- si el chico está practicando
"perro_medias"
 1324      # (decir "per") pero el modelo reconoce algo de OTRA familia
(ej.
 1325      # "perro_correcto", la palabra completa), eso está MAL para este
paso
 1326      # puntual aunque técnicamente haya sonado bien esa otra cosa --
por
 1327      # algo el camino separa los pasos, no sirve de nada si decir la
 1328      # palabra entera "aprueba" el paso del sonido suelto. Sin este
chequeo,
 1329      # el veredicto de abajo solo miraba si la clase ganadora
```

```
terminaba en
 1330      # "_correcto", sin importar de qué familia era.
```

FASE 3B

Modelo de audio: AudioClassifierHead

Red chica sobre el embedding de wav2vec2. Es tu modelo entrenado.

ENTRA	SALE
Embedding normalizado (1, 1024)	Probabilidades de 66 clases, filtradas a la palabra pedida

Hidden: **128** · Dropout: **0.4** · Épocas: **150** · lr: **1e-3** · Weight decay: **1e-3**

Paciencia: **15** · Validación: **20% estratificado** · Semilla: **42** · Clases: **66** · val_acc: **97.68%**

Capas, en el orden en que pasa el dato

Entrada: embedding normalizado	(1, 1024)	scripts/train_audio_classifier.py:100
↓		
Linear 1024 a 128	(1, 128)	scripts/train_audio_classifier.py:100
↓		
ReLU	(1, 128)	scripts/train_audio_classifier.py:101
↓		
Dropout 0.4	(1, 128)	scripts/train_audio_classifier.py:102
↓		
Linear 128 a 66	(1, 66)	scripts/train_audio_classifier.py:103
↓		
softmax y filtro por palabra	clases de esa palabra	server/main.py:909

Arquitectura y entrenamiento (train_audio_classifier.py)

1

AudioClassifierHead

Tu modelo entrenado

clase scripts/train_audio_classifier.py:95

forward scripts/train_audio_classifier.py:106

```
class AudioClassifierHead(in_dim, hidden_dim, num_classes, dropout)
```

Dos capas lineales con ReLU y dropout en el medio.

```
  142  class AudioClassifierHead(nn.Module):
  143      """Cabeza chica sobre el embedding congelado de wav2vec2 -- con
~200
  144      clips en total y un embedding de 1024, una red más grande solo
  145      sobreajustaría. Dropout + weight_decay como regularización
principal."""
  146
  147      def __init__(self, in_dim, hidden_dim, num_classes, dropout):
  148          super().__init__()
  149          self.net = nn.Sequential(
  150              nn.Linear(in_dim, hidden_dim),
  151              nn.ReLU(),
  152              nn.Dropout(dropout),
  153              nn.Linear(hidden_dim, num_classes),
  154          )
  155
  156      def forward(self, x):
  157          return self.net(x)
```

```
  156      def forward(self, x):
  157          return self.net(x)
```

↓

2 stratified_split

Código / procedimiento

scripts/train_audio_classifier.py:81

uso scripts/train_audio_classifier.py:193

```
def stratified_split(y, val_fraction, seed)
```

Separa 20% de cada clase para validación.

```
128  def stratified_split(y, val_fraction, seed):
129      rng = np.random.RandomState(seed)
130      train_idx, val_idx = [], []
131      for c in np.unique(y):
132          idx = np.where(y == c)[0]
133          rng.shuffle(idx)
134          n_val = max(1, int(round(len(idx) * val_fraction)))
135          val_idx.extend(idx[:n_val])
136          train_idx.extend(idx[n_val:])
137      rng.shuffle(train_idx)
138      rng.shuffle(val_idx)
139      return np.array(train_idx), np.array(val_idx)
```

```
246      train_idx, val_idx = stratified_split(y, VAL_FRACTION, SEED)
247      print(f"Train: {len(train_idx)} | Val: {len(val_idx)}")
248
249      # Normalización simple (media/desvío del set de train) -- ayuda
a que la
250      # red chica converja más estable, nada exótico.
251      mean = X[train_idx].mean(axis=0, keepdims=True)
252      std = X[train_idx].std(axis=0, keepdims=True) + 1e-6
253
254      def to_tensor(idx):
255          xs = (X[idx] - mean) / std
256          return torch.from_numpy(xs).float().to(device),
torch.from_numpy(y[idx]).long().to(device)
257
```

↓

3

main

Código / procedimiento

`scripts/train_audio_classifier.py:168`

```
def main()
```

Recorrido: clips (222), embeddings (243), split (246), media y desvío (251 y 252), red (261), optimizador Adam (262), pérdida (263), épocas (269), guardado (327).

```
  221  def main():
  222      clips_por_clase = find_labeled_clips()
  223      if not clips_por_clase:
  224          print(f"No encontré clips con audio en {SESSIONS_DIR}.
Grabá primero con "
  225                "grabar_video_continuo.py.")
  226          return
  227      print("Clases encontradas:")
  228      for clase, wavs in clips_por_clase.items():
  229          print(f"  {clase}: {len(wavs)} clips")
  230
  231      device = "cuda" if torch.cuda.is_available() else "cpu"
  232      todos_cacheados = all(
  233          item.lower().endswith(".npy") for wavs in
clips_por_clase.values() for item in wavs
  234      )
  235      if todos_cacheados:
  236          print("\nTodos los embeddings ya están cacheados -- no
hace falta cargar wav2vec2.")
  237          model = feature_extractor = None
  238      else:
  239          print(f"\nCargando wav2vec2 para extraer embeddings
(device={device})...")
  240          model, feature_extractor, _vocab, device =
```

↓

4

EarlyStopping

Código / procedimiento

`scripts/train_audio_classifier.py:110`

```
class EarlyStopping
```

Corta si val_loss no mejora en 15 épocas.

```
  160  class EarlyStopping:
  161      """Corta el entrenamiento si val_loss deja de mejorar por
`patience`
  162      epocas seguidas -- mismo criterio que usa
train_landmarks_transformer.py
  163      para el modelo visual."""
  164
  165      def __init__(self, patience=15, min_delta=0.001):
  166          self.patience = patience
  167          self.min_delta = min_delta
  168          self.counter = 0
  169          self.best_loss = None
  170          self.early_stop = False
  171
  172      def __call__(self, val_loss):
  173          if self.best_loss is None or val_loss < self.best_loss -
self.min_delta:
  174              self.best_loss = val_loss
  175              self.counter = 0
  176          else:
  177              self.counter += 1
  178              if self.counter >= self.patience:
  179                  self.early_stop = True
```

↓

5 plot_training_curves / plot_confusion_matrix

Código / procedimiento

curvas scripts/train_audio_classifier.py:129

matriz scripts/train_audio_classifier.py:151

```
plot_confusion_matrix(conf, class_names, val_acc)
```

Guardan las gráficas en outputs/.

```
  182  def plot_training_curves(history):
  183      epochs = range(1, len(history["train_loss"]) + 1)
  184      fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
  185
  186      ax1.plot(epochs, history["train_loss"], "b-", label="Train
Loss", linewidth=2)
  187      ax1.plot(epochs, history["val_loss"], "r--", label="Validation
Loss", linewidth=2)
  188      ax1.set_title("Perdida (audio)")
  189      ax1.set_xlabel("Epocas"); ax1.set_ylabel("Perdida");
ax1.legend(); ax1.grid(True)
  190
  191      ax2.plot(epochs, history["train_acc"], "b-", label="Train
Accuracy", linewidth=2)
  192      ax2.plot(epochs, history["val_acc"], "g-", label="Validation
Accuracy", linewidth=2)
  193      ax2.set_title("Precision del clasificador de audio")
  194      ax2.set_xlabel("Epocas"); ax2.set_ylabel("Precision (%)");
ax2.legend(); ax2.grid(True)
  195
  196      plt.tight_layout()
  197      out_path = os.path.join(OUTPUTS_DIR,
"audio_classifier_curvas.png")
  198      os.makedirs(OUTPUTS_DIR, exist_ok=True)
```

```
  204  def plot_confusion_matrix(conf, class_names, val_acc):
  205      n = len(class_names)
  206      fig, ax = plt.subplots(figsize=(max(6, n * 0.45), max(5, n *
0.45)))
  207      im = ax.imshow(conf, cmap="Blues")
  208      ax.set_xticks(range(n)); ax.set_xticklabels(class_names,
rotation=90, fontsize=8)
  209      ax.set_yticks(range(n)); ax.set_yticklabels(class_names,
fontsize=8)
  210      ax.set_xlabel("Predicho"); ax.set_ylabel("Real")
  211      ax.set_title(f"Matriz de confusion (audio) -- val acc
{val_acc*100:.1f}%")
  212      plt.colorbar(im)
  213      plt.tight_layout()
  214      out_path = os.path.join(OUTPUTS_DIR,
"audio_matriz_confusion.png")
```

```
215     os.makedirs(OUTPUTS_DIR, exist_ok=True)
216     plt.savefig(out_path)
217     plt.close()
218     print(f"-> Matriz de confusion guardada: {out_path}")
```

↓

6

Guardado del modelo

Código / procedimiento

`scripts/train_audio_classifier.py:272`

```
torch.save({state_dict, class_names, embedding_dim, hidden_dim, mean,
std, val_acc})
```

Guarda models/audio_classifier.pth con la media y el desvío incluidos.

```
327     torch.save(
328         {
329             "state_dict": best_state,
330             "class_names": class_names,
331             "embedding_dim": EMBEDDING_DIM,
332             "hidden_dim": HIDDEN_DIM,
333             "mean": mean,
334             "std": std,
335             "val_acc": best_val_acc,
336         },
337         MODEL_OUT,
338     )
```

En el servidor

1

Carga

Código / procedimiento

`server/main.py:522 a 623`

`AudioClassifierHead(...); clf.load_state_dict(ckpt['state_dict'])`

Se carga al arrancar, junto con los nombres de clase, la media (621) y el desvío (622).

```
  613             clf = AudioClassifierHead(
  614                 ckpt["embedding_dim"], ckpt["hidden_dim"],
len(ckpt["class_names"]), dropout=0.0,
  615             ).to(DEVICE)
  616             clf.load_state_dict(ckpt["state_dict"])
  617             clf.eval()
  618             _state.update(
  619                 audio_clf=clf,
  620                 audio_clf_classes=ckpt["class_names"],
  621                 audio_clf_mean=ckpt["mean"],
  622                 audio_clf_std=ckpt["std"],
  623             )
  624             print(f"Clasificador de audio entrenado cargado
({len(ckpt['class_names'])} clases, "
```

↓

2 _classify_audio Código / procedimiento

`server/main.py:875`

```
def _classify_audio(audio_path, palabra)
```

Embedding (1105), normalización (1106), red y softmax (1113). Devuelve clase predicha, probabilidad y es_correcto.

```
 1070  def _classify_audio(audio_path, palabra):
 1071      """Clasificador de audio ENTRENADO con clips reales (ver
 1072      train_audio_classifier.py) -- a diferencia del GOP de arriba
(que fuerza
 1073      el audio contra la palabra esperada pase lo que pase), esto
predice
 1074      directamente entre las clases que existan para esa palabra
 1075      ("<palabra>_correcto", "<palabra>_incorrecto_*"), así que SÍ
puede
 1076      reconocer que el audio no se parece a nada de lo esperado.
 1077
 1078      Devuelve {"clase_predicha", "prob", "es_correcto"} o None si
no hay
 1079      clasificador entrenado, o si esa palabra no tiene clases
entrenadas
 1080      todavía (el clasificador solo conoce las palabras con clips
grabados)."""
 1081      clf = _state["audio_clf"]
 1082      if clf is None or audio_path is None or not palabra:
 1083          return None
 1084      class_names = _state["audio_clf_classes"]
 1085      # Solo tiene sentido clasificar si HAY clases de esta palabra
entre las
 1086      # que el clasificador aprendió -- si pedís "perro" pero solo
```

↓

3 Filtro por palabra

Código / procedimiento

lista `server/main.py:894` restricción `server/main.py:909`

```
indices_de_la_palabra; probs_familia = probs[indices_de_la_palabra]
```

Se queda solo con palabra_correcto y palabra_incorrecto. La probabilidad que devuelve es la del softmax global, por eso sale baja si el audio no se parece a nada de esa palabra.

```
 1098      indices_de_la_palabra = [
 1099          i for i, c in enumerate(class_names)
 1100          if c == f"{palabra}_correcto" or c.startswith(f"
{palabra}_incorrecto")
 1101      ]
 1102      if not indices_de_la_palabra:
 1103          return None
 1104      try:
 1105          emb = extract_embedding(audio_path, _state["audio_model"],
_state["audio_feature_extractor"], DEVICE)
 1106          emb_norm = (emb - _state["audio_clf_mean"]) /
_state["audio_clf_std"]
 1107          # mean/std se guardaron con shape (1, 1024) (ver
train_audio_
 1108          # classifier.py) -- emb_norm ya sale (1, 1024) por el
broadcasting,
 1109          # así que reshape en vez de unsqueeze (que agregaba una
dimensión de
```

```
 1123          probs_familia = probs[indices_de_la_palabra]
 1124          idx_local = int(torch.argmax(probs_familia).item())
 1125          idx = indices_de_la_palabra[idx_local]
 1126          clase_predicha = class_names[idx]
 1127          return {
 1128              "clase_predicha": clase_predicha,
 1129              "prob": float(probs[idx].item()),
 1130              "es_correcto": clase_predicha == f"{palabra}_correcto",
 1131          }
 1132      except Exception as e:
 1133          print(f"[AVISO] Falló el clasificador de audio entrenado
({e}) -- sigo solo con el GOP.")
 1134          return None
```

↓

4 Quién la llama

Código / procedimiento

solo voz `server/main.py:925` con cámara `server/main.py:1150`

frase `server/main.py:1304`

```
_classify_audio(audio_path, palabra)
```

Los tres caminos usan la misma función.

```
 1156      audio_clf_resultado = _classify_audio(audio_path,
palabra_esperada)
 1157      prob_str = f"{audio_clf_resultado['prob']:.2f}" if
audio_clf_resultado else "?"
 1158      print(f"[predict-audio] audio GOP {'OK' if audio_resultado is
None else ('bien' if audio_ok else 'sospechoso')} | "
 1159            f"clasificador {'sin entrenar' if audio_clf_resultado is
None else audio_clf_resultado['clase_predicha']} "
 1160            f"prob={prob_str} (umbral {AUDIO_MIN_CONFIDENCE}) "
 1161            f"({time.time() - t_start:.1f}s total)")
 1162
 1163      if audio_clf_resultado is None:
 1164          return {
 1165              "valid": False,
 1166              "reason": "no se pudo analizar el audio -- probá de
nuevo",
 1167          }
```

```
 1396      audio_clf_resultado = _classify_audio(audio_path,
palabra_esperada or palabra)
 1397      prob_str = f"{audio_clf_resultado['prob']:.2f}" if
audio_clf_resultado else "?"
 1398      print(f"[predict] audio GOP {'OK' if audio_resultado is None
else ('bien' if audio_ok else 'sospechoso')} | "
 1399            f"clasificador {'sin entrenar' if audio_clf_resultado is
None else audio_clf_resultado['clase_predicha']} "
 1400            f"prob={prob_str} (umbral {AUDIO_MIN_CONFIDENCE})
video_prob={top_prob:.2f} "
 1401            f"({time.time() - t_start:.1f}s total)")
 1402
 1403      # NOTA histórica: acá antes había un esquema de "prioridad" (el
audio
 1404      # mandaba solo si estaba seguro, el video era respaldo) --
funcionaba
 1405      # bien pero el docente pidió específicamente la matriz cruzada
de abajo,
 1406      # que evalúa las dos señales por separado y exige que confirmen
las dos.
 1407      #
=====================================================================
```

```
 1623          clf_resultado = _classify_audio(audio_recortado_path,
palabra) if audio_recortado_path else None
 1624
 1625          gop_ok = len(resultado["sospechosos"]) == 0 and
resultado["palabra_reconocida"]
 1626
 1627          # BUG real detectado (caso "gorra" -> "goda"): sacar el
umbral de
 1628          # confianza del todo (commit anterior, por el caso
"sherpiente") fue
 1629          # un arreglo mal calibrado -- "sherpiente" en realidad tenía
80% de
 1630          # confianza real (muy por encima del umbral), así que nunca
hacía
 1631          # falta sacarlo. Sin el umbral, el clasificador también
"decidía"
 1632          # con 0-2% de confianza (el audio recortado no se parece a
NADA
 1633          # entrenado -- típico cuando el recorte de la frase sale
raro) y
 1634          # igual contaba "bien" solo porque esa clase ganó por aunque
sea un
```

FASE 3C

GOP: pronunciación por fonema

Procedimiento sobre wav2vec2. No es un modelo entrenado por vos: alinea los fonemas esperados contra el audio y dice cuáles suenan dudosos.

ENTRA	SALE
WAV y palabra (o frase y palabra objetivo)	Fonemas, score por fonema, dudosos, lo reconocido y tipo de error

Umbral de dudoso: **-3.0 (aud:52)**

Palabra no reconocida: **distancia > 0.5 (aud:63)**

Frase entera: **≥ 60% (aud:556)**

Frase pedida: **menos de 2 dudosos fuera**

Palabra sola: score_pronunciation

1 score_pronunciation

Código / procedimiento

scripts/audio_pronunciation.py:296 envoltorio server/main.py:856

```
def score_pronunciation(wav_path, palabra, model, feature_extractor,
vocab, device)
```

Orquesta todos los pasos de abajo y arma el resultado.

```
  454  def score_pronunciation(wav_path, palabra, model,
feature_extractor, vocab, device):
  455      """Puntúa la pronunciación de `palabra` en el clip `wav_path`.
  456
  457      Devuelve un dict:
  458          {
  459              "fonemas": ["p", "e", "r", "o"],
  460              "scores": [-0.12, -0.34, -2.81, -0.09],   # log-prob
promedio, uno por fonema
  461              "sospechosos": [2],                        # índices
por debajo del umbral
  462              "reconocido_libre": "p e r o",              # qué
reconoció el modelo SIN forzar la palabra
  463          }
  464      """
  465      waveform = load_audio_16k(wav_path)
  466
  467      inputs = feature_extractor(waveform.numpy(),
sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
  468      with torch.inference_mode():
  469          logits = model(inputs.input_values.to(device)).logits  #
(1, T, C)
  470      log_probs = torch.log_softmax(logits, dim=-1)
  471
```

```
 1035  def _score_audio_pronunciation(audio_path, palabra):
 1036      """Envoltorio fino sobre
audio_pronunciation.score_pronunciation() --
 1037      nunca tira: si el audio no se pudo leer/puntuar, devuelve None y
el
 1038      veredicto final cae solo en el video (ver
_run_prediction_pipeline)."""
 1039      if audio_path is None or not palabra:
 1040          return None
 1041      if _state["audio_model"] is None:
 1042          return None
 1043      try:
 1044          return score_pronunciation(
 1045              audio_path, palabra,
 1046              _state["audio_model"],
_state["audio_feature_extractor"], _state["audio_vocab"], DEVICE,
 1047          )
```

```
 1048      except Exception as e:
 1049          print(f"[AVISO] Falló el análisis de audio ({e}) -- sigo
solo con video.")
 1050          return None
```

↓

2

texto_a_fonemas

Código / procedimiento

`scripts/audio_pronunciation.py:22`

```
def texto_a_fonemas(palabra)
```

Reglas del español: rr y r inicial dan r, r suave da ɾ, ch da tʃ, ll da ʝ, c y z dan s, la h es muda.

```
   82  def texto_a_fonemas(palabra):
   83      """'perro' -> ['p','e','r','o']. 'carro' -> ['k','a','r','o'].
   84      'flor' -> ['f','l','o','ɾ'] (r suave, no vibrante, porque no
está al
   85      principio de palabra ni duplicada)."""
   86      palabra = palabra.strip().lower()
   87      for acentuada, simple in VOCALES_ACENTUADAS.items():
   88          palabra = palabra.replace(acentuada, simple)
   89
   90      fonemas = []
   91      i = 0
   92      n = len(palabra)
   93      while i < n:
   94          c = palabra[i]
   95          nxt = palabra[i + 1] if i + 1 < n else ""
   96          prev = palabra[i - 1] if i > 0 else ""
   97
   98          if c == "r":
   99              doble = nxt == "r"
  100              al_inicio = i == 0
  101              tras_nls = prev in ("n", "l", "s")
  102              fonemas.append("r" if (doble or al_inicio or tras_nls)
else "ɾ")
  103              i += 2 if doble else 1
```

↓

3

_phoneme_to_token_ids

Código / procedimiento

`scripts/audio_pronunciation.py:176`

```
def _phoneme_to_token_ids(fonemas, vocab)
```

Pasa cada fonema a su número en el vocabulario del modelo.

```
  283  def _phoneme_to_token_ids(fonemas, vocab):
  284      """Mapea cada símbolo IPA de texto_a_fonemas() a su id en el
vocabulario
  285      del modelo. Si algún fonema no está en el vocabulario (no
debería pasar
  286      con los símbolos que usa texto_a_fonemas(), todos estándar), se
avisa y
  287      se saltea -- mejor perder un fonema puntual que romper todo el
score."""
  288      ids = []
  289      fonemas_usados = []
  290      for f in fonemas:
  291          if f in vocab:
  292              ids.append(vocab[f])
  293              fonemas_usados.append(f)
  294          else:
  295              print(f"[AVISO] Fonema '{f}' no está en el vocabulario
del modelo, se saltea.")
  296      return ids, fonemas_usados
```

↓

4 forced_align

Modelo externo ya entrenado

`scripts/audio_pronunciation.py:317`

```
F.forced_align(log_probs, targets, blank=blank_id)
```

Alineación CTC de torchaudio: ubica cada fonema esperado en el audio y da su score medio.

```
  487      aligned_labels, aligned_scores = F.forced_align(
  488          log_probs, targets, blank=blank_id,
  489      )
  490      aligned_labels = aligned_labels[0].tolist()
  491      aligned_scores = aligned_scores[0].tolist()
  492
  493      # Agrupa los frames alineados por fonema (cada fonema ocupa
varios
  494      # frames consecutivos) y promedia el score dentro de cada grupo.
  495      scores_por_fonema = []
  496      idx_target = 0
  497      frame_scores_actual = []
  498      for label, score in zip(aligned_labels, aligned_scores):
```

↓

5

_greedy_ctc_decode

Código / procedimiento

`scripts/audio_pronunciation.py:188`

```
def _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)
```

Reconocimiento libre: qué fonemas sonaron sin forzar la palabra.

```
  299  def _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id):
  300      """Decodificación CTC libre (sin forzar ninguna palabra) --
colapsa
  301      repeticiones consecutivas y saca el blank, igual que hace
  302      processor.batch_decode() normalmente. Devuelve la lista de
símbolos
  303      (no un string) porque además de mostrarse en el reporte, se usa
para
  304      comparar contra los fonemas esperados (ver
_distancia_edicion)."""
  305      fonemas = []
  306      anterior = None
  307      for tid in pred_ids:
  308          if tid != anterior and tid != blank_id:
  309              fonemas.append(id_to_phoneme.get(tid, "?"))
  310          anterior = tid
  311      return fonemas
```

↓

6

_distancia_edicion

Código / procedimiento

`scripts/audio_pronunciation.py:205`

```
def _distancia_edicion(a, b)
```

Distancia entre lo esperado y lo reconocido. Si pasa de 0.5 se considera que no dijo la palabra.

```
  326  def _distancia_edicion(a, b):
  327      """Distancia de Levenshtein entre dos listas de símbolos --
cuántas
  328      inserciones/borrados/sustituciones hacen falta para convertir
`a` en
  329      `b`. Programación dinámica clásica, O(len(a)*len(b))."""
  330      n, m = len(a), len(b)
  331      dp = [[0] * (m + 1) for _ in range(n + 1)]
  332      for i in range(n + 1):
  333          dp[i][0] = i
  334      for j in range(m + 1):
  335          dp[0][j] = j
  336      for i in range(1, n + 1):
  337          for j in range(1, m + 1):
  338              if a[i - 1] == b[j - 1]:
  339                  dp[i][j] = dp[i - 1][j - 1]
  340              else:
  341                  dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i
- 1][j - 1])
  342      return dp[n][m]
```

↓

7 diagnose_tipo_error

Código / procedimiento

`scripts/audio_pronunciation.py:268`

tablas `scripts/audio_pronunciation.py:256 y 411`

```
def diagnose_tipo_error(fonemas_esperados, reconocido_libre_lista)
```

Usa _align_ops (línea 357) para ver qué se sustituyó. Ej.: r por l es lambdacismo, s por θ es interdental.

```
  414  def diagnose_tipo_error(fonemas_esperados,
reconocido_libre_lista):
  415      """Devuelve un tipo de error (omision / lambdacismo /
dentalizacion /
  416      anteriorizacion / debilitamiento / interdental / oclusion /
epentesis)
  417      o None, mirando qué pasó en la posición del fonema difícil de
la
  418      palabra dentro de la alineación entre lo esperado y lo
reconocido
  419      libremente. Cubre los 5 sonidos que el proyecto entrena hoy
(R/RR, S,
  420      G, C/K, Sinfones con L) -- si la palabra no tiene ninguno de
esos
  421      fonemas, o el error no encaja en ningún patrón conocido,
devuelve None
  422      en vez de forzar una etiqueta."""
  423      ops = _align_ops(fonemas_esperados, reconocido_libre_lista)
  424
  425      # Sinfones (bl/cl/fl/gl/pl): el error típico es comerse la "l"
del grupo
  426      # (reducción) o meter una vocal en el medio (epéntesis) -- se
busca una
  427      # "l" esperada que venga justo después de una consonante de
```

```
  402  _SUSTITUCIONES_CONOCIDAS = {
  403      "r": {"l": "lambdacismo", "d": "dentalizacion"},
  404      "ɾ": {"l": "lambdacismo", "d": "dentalizacion"},
  405      "k": {"t": "anteriorizacion", "x": "debilitamiento", "h":
"debilitamiento"},
  406      "g": {"d": "anteriorizacion", "x": "debilitamiento", "h":
"debilitamiento"},
  407      "s": {"θ": "interdental", "t": "oclusion", "d": "oclusion"},
  408      "ʝ": {"l": "distorsion", "i": "debilitamiento"},
  409      "tʃ": {"ʃ": "distorsion", "t": "debilitamiento", "s":
"debilitamiento"},
  410  }
  411  _SINFONES = {"bl", "kl", "fl", "gl", "pl"}  # cl -> "k"+"l" en
fonemas (texto_a_fonemas usa "k")
```

```
  412
  413
```

```
  411  _SINFONES = {"bl", "kl", "fl", "gl", "pl"}  # cl -> "k"+"l" en
fonemas (texto_a_fonemas usa "k")
  412
  413
  414  def diagnose_tipo_error(fonemas_esperados, reconocido_libre_lista):
  415      """Devuelve un tipo de error (omision / lambdacismo /
dentalizacion /
  416      anteriorizacion / debilitamiento / interdental / oclusion /
epentesis)
  417      o None, mirando qué pasó en la posición del fonema difícil de la
  418      palabra dentro de la alineación entre lo esperado y lo
reconocido
  419      libremente. Cubre los 5 sonidos que el proyecto entrena hoy
(R/RR, S,
  420      G, C/K, Sinfones con L) -- si la palabra no tiene ninguno de
esos
  421      fonemas, o el error no encaja en ningún patrón conocido,
devuelve None
  422      en vez de forzar una etiqueta."""
```

↓

8

fonemas_a_texto_aproximado

Código / procedimiento

`scripts/audio_pronunciation.py:201`

```
def fonemas_a_texto_aproximado(fonemas)
```

Texto aproximado de lo que sonó.

```
  321  def fonemas_a_texto_aproximado(fonemas):
  322      """['p','e','l','o'] -> 'pelo'. Solo para mostrar, no para
comparar."""
  323      return "".join(_FONEMA_A_LETRA.get(f, f) for f in fonemas)
```

Frase completa: score_word_in_sentence

1

score_word_in_sentence

Código / procedimiento

`scripts/audio_pronunciation.py:385` envoltorio `server/main.py:864`

```
def score_word_in_sentence(wav_path, frase, palabra_objetivo, model,
feature_extractor, vocab, device)
```

Alinea la frase entera (forced_align en la línea 621), ubica la palabra, calcula los dos filtros y recorta su audio con 150 ms de margen.

```
  586  def score_word_in_sentence(wav_path, frase, palabra_objetivo,
model, feature_extractor, vocab, device):
  587      """Como score_pronunciation(), pero el chico dice una FRASE
completa
  588      (ej. "el perro corre por las montañas") en vez de la palabra
sola --
  589      pedido explícito: que se pueda practicar en un contexto más
natural,
  590      no solo palabras sueltas. No usa el clasificador de audio
entrenado
  591      (ver train_audio_classifier.py) porque ESE se entrenó
únicamente con
  592      clips de la palabra sola: si se le da una frase entera, el
embedding
  593      resume toda la frase junta y la clasificación no tiene
sentido. Esto
  594      en cambio solo necesita el GOP genérico (wav2vec2 sin
entrenamiento
  595      propio), que no depende de ningún dataset -- se alinea la
frase
  596      ENTERA contra el audio, y después se recorta el resultado solo
al
  597      tramo donde aparece `palabra_objetivo` dentro de esa frase.
  598
```

```
 1053  def _score_word_in_sentence(audio_path, frase, palabra):
 1054      """Envoltorio fino sobre
audio_pronunciation.score_word_in_sentence() --
 1055      nunca tira: si falla, devuelve None."""
 1056      if audio_path is None or not frase or not palabra:
 1057          return None
 1058      if _state["audio_model"] is None:
 1059          return None
 1060      try:
 1061          return score_word_in_sentence(
 1062              audio_path, frase, palabra,
 1063              _state["audio_model"],
_state["audio_feature_extractor"], _state["audio_vocab"], DEVICE,
 1064          )
 1065      except Exception as e:
```

```
 1066          print(f"[AVISO] Falló el análisis de frase ({e}).")
 1067          return None
```

↓

2

_localizar_subsecuencia

Código / procedimiento

`scripts/audio_pronunciation.py:369`

```
def _localizar_subsecuencia(secuencia, objetivo)
```

Busca dónde cae la palabra dentro de la frase por distancia de edición.

```
  559  def _localizar_subsecuencia(secuencia, objetivo):
  560      """Busca dónde aparece `objetivo` (lista de fonemas de la
palabra sola)
  561      dentro de `secuencia` (lista de fonemas de la frase completa),
probando
  562      todas las ventanas de largo parecido y quedándose con la de
menor
  563      distancia de edición. No exige coincidencia exacta a
propósito: la
  564      regla de "r fuerte al principio de palabra" en
texto_a_fonemas() mira
  565      el inicio del STRING completo, así que una palabra que empieza
con r
  566      en medio de una frase puede fonemizarse levemente distinto que
la
  567      misma palabra calculada sola -- por eso se busca la MEJOR
ventana, no
  568      un substring exacto.
  569
  570      Devuelve (inicio, fin) del mejor tramo dentro de
`secuencia`."""
  571      n, m = len(secuencia), len(objetivo)
  572      if m == 0 or n == 0:
  573          return 0, 0
```

↓

3 Filtros de la frase

Código / procedimiento

entera `scripts/audio_pronunciation.py:366` pedida `server/main.py:1297`

`frase_completa_dicha; resto_de_la_frase_ok`

El largo de lo reconocido debe ser al menos el 60% de lo esperado, y debe haber menos de 2 fonemas dudosos fuera de la palabra.

```
  556  FRASE_COMPLETA_LARGO_MINIMO = 0.6
  557
  558
  559  def _localizar_subsecuencia(secuencia, objetivo):
  560      """Busca dónde aparece `objetivo` (lista de fonemas de la
palabra sola)
  561      dentro de `secuencia` (lista de fonemas de la frase completa),
probando
  562      todas las ventanas de largo parecido y quedándose con la de
menor
  563      distancia de edición. No exige coincidencia exacta a propósito:
la
  564      regla de "r fuerte al principio de palabra" en texto_a_fonemas()
mira
  565      el inicio del STRING completo, así que una palabra que empieza
con r
  566      en medio de una frase puede fonemizarse levemente distinto que
la
  567      misma palabra calculada sola -- por eso se busca la MEJOR
ventana, no
```

```
 1616          if not resultado.get("resto_de_la_frase_ok", True):
 1617              return {
 1618                  "valid": False,
 1619                  "frase_distinta": True,
 1620                  "reason": "repetí la frase de arriba tal cual --
¡dale, de nuevo!",
 1621              }
 1622
 1623          clf_resultado = _classify_audio(audio_recortado_path,
palabra) if audio_recortado_path else None
 1624
 1625          gop_ok = len(resultado["sospechosos"]) == 0 and
resultado["palabra_reconocida"]
 1626
 1627          # BUG real detectado (caso "gorra" -> "goda"): sacar el
umbral de
```

Probarlo solo

1

print_report

Código / procedimiento

`scripts/audio_pronunciation.py:501`

```
python scripts/audio_pronunciation.py <archivo.wav> <palabra>
```

Imprime el reporte fonema por fonema. Sirve para mostrar el GOP en vivo.

```
  750  def print_report(palabra, resultado):
  751      print(f"\nPalabra objetivo: '{palabra}'")
  752      print(f"Fonemas esperados: {' '.join(resultado['fonemas'])}")
  753      print(f"Lo que reconoció el modelo (libre, sin forzar):
{resultado['reconocido_libre']}")
  754      print("Score por fonema (más alto = más claro, más bajo = más
dudoso):")
  755      for i, (fon, score) in enumerate(zip(resultado["fonemas"],
resultado["scores"])):
  756          marca = " <-- sospechoso" if i in resultado["sospechosos"]
else ""
  757          score_str = f"{score:.2f}" if score is not None else "N/A"
  758          print(f"  {fon:>4s}: {score_str}{marca}")
  759      if resultado["sospechosos"]:
  760          letras = [resultado["fonemas"][i] for i in
resultado["sospechosos"]]
  761          print(f"\n-> Posible error de pronunciación en: {',
'.join(letras)}")
  762      else:
  763          print("\n-> Todos los fonemas dentro de lo esperado.")
  764      if not resultado["palabra_reconocida"]:
  765          print(f"-> OJO: lo reconocido libremente se parece poco a
'{palabra}' "
  766                f"(distancia normalizada
```

FASE 2C · CARA COMPLETA (EL ÚNICO MODELO DE VIDEO DE LA APP)

Landmarks de toda la cara

Hasta ahora el modelo de video solo veía los **20 puntos de los labios**. Este modelo le da **68 puntos de toda la cara** (contorno y mandíbula, cejas, nariz, ojos y boca), para que también aprenda del movimiento de la mandíbula y de la cara, como pidió el docente.

ENTRA
Video (.avi o .webm) con T frames

SALE
.npy de (T, 408) por clip, en data/landmarks_cara_completa_npy/<clase>/

Puntos: **68 (índices 0 a 67)** · Referencia: **ojos 36 y 45** · Por frame: **408 = 136 + 136 + 136**

Salto máximo: **0.15** · Suavizado alpha: **0.4** · Mínimo con cara: **50 % de los frames**

Clases del modelo: **23 (blanco, burro, carro, casa, cama, clavo, cohete, copa, cubo y flor)**

Qué cambia respecto al modelo actual

	Actual (solo boca)	Modelo final (cara completa)
Puntos	20 (labios, índices 48-67)	68 (índices 0-67)
Referencia para normalizar	Comisuras de la boca (48 y 54)	Esquinas externas de los ojos (36 y 45)
Números por frame	120 (40 posición + 40 velocidad + 40 aceleración)	408 (136 + 136 + 136)
Clips con la cara cortada	Se guardan igual (basta ver la boca)	Se descartan si la cara se ve en menos del 50 % de los frames
Carpeta de salida	data/landmarks_npy/	data/landmarks_cara_completa_npy/
Modelo que se entrena	LipReadingConformer, input_dim 120	La misma red, input_dim 408 (se detecta sola desde el .npy)
Dónde queda el modelo	models/best.pth (modelo de labios; desactivado en el servidor)	data/modelos_cara_completa/best_cara_completa.pth (el que usa la app)

Recorrido

1

Buscar los videos y contar por clase

Código / procedimiento

extracción `Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 3`

```
clase_del_video(ruta)
```

Recorre data/sesiones_continuas y dataARCHIVADO, toma .avi y .webm, y los agrupa en las 58 clases. Imprime cuántos videos hay por clase junto a cuántos landmarks viejos había, para ver antes de empezar si faltan videos.

↓

2

Detector HRNet / FAN

Modelo externo ya entrenado

extracción `Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 4`

```
build_hrnet_detector()  ·  detect_hrnet(landmarker, frame_bgr)
```

El mismo detector de siempre (paquete face-alignment): da 68 puntos (x, y) por frame, o None si no encuentra la cara. Esta vez se usan los 68 y no solo los de los labios.

↓

3

Normalizar la cara completa

Código / procedimiento

extracción `Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 4`

```
normalize_face_landmarks(landmarks_px)
```

Centro = punto medio entre las esquinas externas de los ojos (36 y 45); escala = distancia entre ellas; rotación = para dejarlas horizontales. Así la mandíbula y la boca se miden contra la parte rígida de la cara. Devuelve 136 números (68 × 2).

↓

4 Completar, suavizar y calcular velocidad y aceleración

Código / procedimiento

extracción `Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 4`

```
fill_missing_frames · smooth_positions · add_dynamics · process_clip
```

Idéntico al modelo de solo boca: se rellenan los frames sin cara, se suaviza (alpha 0.4) y se calculan velocidad y aceleración. La salida son 136 + 136 + 136 = 408 números por frame.

↓

5 Filtro de calidad: ¿se ve la cara completa? Código / procedimiento

extracción `Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 5`

```
MIN_FRACCION_CON_CARA = 0.5
```

Si el detector no encuentra la cara en al menos la mitad de los frames (por ejemplo, la cámara corta la cara y no se ven los ojos), el clip se descarta y queda anotado en descartados_sin_cara_completa.json. Sin este filtro se guardaría una posición repetida, que no es movimiento. Se puede poner 0 para guardar todo.

↓

6 Entrenar con los landmarks de cara completa Tu modelo entrenado

entrenamiento `Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celdas 3 a 5`

```
LipReadingConformer(num_classes=58, input_dim=408)
```

Es exactamente la misma red (TCN + Conformer) y los mismos hiperparámetros (50 épocas completas, sin parada temprana, lote 32, 60 frames, lr 1e-3, label smoothing 0.1, semilla 42 y 80/20). Lo único que cambia es el tamaño de la entrada, que se detecta solo desde el .npy. Guarda en data/modelos_cara_completa/, que es lo que carga la app; el models/best.pth (labios) ya no se usa en el servidor.

↓

7

Comparar contra el modelo de solo labios

Código / procedimiento

entrenamiento Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 7

```
Top-1 · Top-3 · bien o mal
```

Al terminar imprime una tabla con Top-1, Top-3 y acierto bien/mal del modelo nuevo junto a las cifras del modelo actual (67,17 % / 92,46 % / 75,98 %) y la diferencia. También guarda las curvas, la matriz de confusión y un JSON de resultados.

↓

8

Entrenamiento real en la PC (54 clases)

Tu modelo entrenado

entrenamiento Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celdas 3 a 7

```
LipReadingConformer(num_classes=54, input_dim=408)
```

Con los 10.917 clips extraídos (54 clases con datos de 23 palabras; falta perro): 8.739 para entrenar y 2.178 para validar. Se entrenaron las 50 épocas completas, sin parada anticipada (con paciencia de 15 un intento se detuvo en la época 27 con 64 % porque la pérdida de validación salta mucho al inicio). Resultado: Top-1 89,67 %, Top-3 99,17 % y bien o mal 90,36 %. Para compararlo justo se entrenó el modelo de labios con los mismos clips, las mismas clases y la misma división (los 20 puntos de la boca sacados de los mismos archivos): Top-1 85,17 %, Top-3 98,76 % y bien o mal 86,69 %. La cara completa queda 4,5 puntos arriba en Top-1 (IC 95 %: 3,0 a 5,9; McNemar $p < 0{,}001$) y 3,7 en bien o mal ($p < 0{,}001$); en Top-3 la diferencia no es significativa ($p = 0{,}14$). Salió de una sola corrida con una semilla. El reparto sale de las mismas sesiones y voces.

↓

9 El servidor elige qué modelo de video usar

Código / procedimiento

servidor `server/main.py:1049` carga `server/main.py:454`

normalización `server/main.py:171`

```
modelo_video = 'cara_completa'
```

Si la palabra pedida tiene sus dos clases (_correcto e _incorrecto) en el modelo de cara completa, el servidor normaliza los 68 puntos con los ojos y, si la cara se ve entera en al menos el 60 % de los frames, usa ese modelo; si no, pide repetir la toma. El modelo de labios quedó desactivado (comentado en el código con la marca [LABIOS DESACTIVADO]). Si la palabra no está en el modelo de cara completa (hoy, perro), se evalúa solo con el audio. La respuesta trae modelo_video = cara_completa.

↓

10 Lo que se ve en la pantalla

Código / procedimiento

web `web/src/screens/HomeScreen.jsx:39`

```
gruposDeLaCara()  ·  dibujarPuntos(ctx, cara, indices, color, radio, ancho, alto)
```

La cámara dibuja sobre la cara, con MediaPipe en el navegador: contorno y mandíbula (violeta), cejas (naranja), ojos (celeste), labios (verde), mejillas (rosado) y nariz (blanco). Es solo visual: quien decide es el servidor.

Estado: el modelo de cara completa se entrenó en la PC con 54 clases y está conectado al servidor, y es el único modelo de video del servidor (el de labios quedó desactivado y comentado); perro no está en ese conjunto y se evalúa solo con audio. En la comparación justa (mismos 10.917 clips, mismas clases, misma división) la cara completa acertó 89,67 % y los labios 85,17 %: **la mejora es estadísticamente significativa (+4,5 puntos; p < 0,001)**, aunque proviene de una sola corrida y de voces compartidas con el entrenamiento.

Preguntas que te pueden hacer

¿Qué cambió respecto a antes?

Antes el modelo recibía 20 puntos de los labios. Ahora recibe 68 puntos de toda la cara (contorno y mandíbula, cejas, nariz, ojos y boca), medidos contra los ojos. La red es la misma.

¿Por qué se normaliza con los ojos y no con la boca?

Porque ahora también se mide la mandíbula y la boca: necesitan una referencia que no se mueva al hablar. Las esquinas de los ojos casi no se mueven; las de la boca sí.

¿Por qué se descartan clips?

Algunos clips se grabaron con la cámara tan baja que no se ven los ojos. El detector no encuentra la cara completa y lo que quedaría es una posición repetida, que no sirve para entrenar. El notebook lista cuántos se descartan por clase. Por eso, en la app, si no se ven los ojos el servidor pide repetir la toma.

¿Incluye las mejillas?

El detector de 68 puntos da el contorno de la cara (mandíbula y borde de las mejillas), no puntos dentro de las mejillas. Para más detalle se podría usar MediaPipe con 478 puntos; sería otro experimento.

¿Ya mejoró el modelo?

Sí. Con los mismos 10.917 clips y las mismas clases, la cara completa dio 89,67 % y los labios 85,17 % (top-3: 99,17 % y 98,76 %; bien o mal: 90,36 % y 86,69 %). La diferencia en el acierto exacto (4,5 puntos) es estadísticamente significativa (prueba de McNemar exacta, $p < 0{,}001$; IC 95 % de la diferencia: 3,0 a 5,9), pero salió de una sola corrida y de voces compartidas entre entrenamiento y validación, así que puede ser optimista. Falta repetirla con otras semillas, con voces nuevas y con la palabra perro.

¿Ya está en la app?

Sí. El servidor lo carga al arrancar y es su único modelo de video, para las 23 palabras con datos (todas menos perro) cuando se ven los ojos; si no se ven, pide repetir la toma, y perro se evalúa solo con audio.

¿Qué se ve en pantalla?

La cámara dibuja el contorno de la cara y la mandíbula, las cejas, los ojos, los labios, las mejillas y la nariz. Es solo visual; el servidor decide con el modelo.

¿Qué limitaciones tiene?

Necesita los videos originales (ya se extrajeron 10.917 clips en 54 clases; falta la palabra perro). Los clips con la cara cortada se pierden. Y el detector es lento: unos 14 s por clip en la PC, porque procesa un frame a la vez.

FASE 4 · MAPA DEL CÓDIGO

Dónde se usan los modelos

Para cada modelo: dónde se define, dónde se entrena, dónde se guarda, dónde se carga el servidor y dónde se usa al evaluar. Los números de línea son los del código actual; con el nombre de la función se vuelve a encontrar con Ctrl+F.

Tu modelo entrenado · Modelo externo ya entrenado · Código / procedimiento

Los cinco modelos de un vistazo

Modelo	Se define / se entrena	Se guarda	El servidor lo carga	El servidor lo usa
HRNet / FAN externo puntos de la cara	No se entrena: paquete face-alignment scripts/extraer_landmarks_npy.py:178	—	server/main.py:479 (server/main.py:89)	server/main.py:1058 (en /predict) server/main.py:105
LipReadingConformer tuyo video: TCN + Conformer	Clase: scripts/train_landmarks_transformer.py:179 Entrena: scripts/train_landmarks_transformer.py:407	scripts/train_landmarks_transformer.py:462 → models/best.pth (scripts/train_landmarks_transformer.py:23)	server/main.py:449 (clase en server/main.py:391)	server/main.py:1131 dentro de _run_prediction_pipeline (server/main.py:1013)
LipReadingConformer (cara completa) tuyo video con 68 puntos, 15 clases	Misma clase: scripts/train_landmarks_transformer.py:179 Entrena: Colab/Entrenar_Cara_Completa_FonoKids.ipynb	data/modelos_cara_completa/best_cara_completa.pth	server/main.py:454	server/main.py:1049 (único modelo de video; perro va solo con audio)
wav2vec2 externo voz a fonemas	No se entrena: facebook/wav2vec2-xlsr-53-espeak-cv-ft scripts/audio_pro	—	server/main.py:515	Embedding: scripts/audio_pronunciation.py:167 GOP: scripts/audio_pro

	nunciation.py:129			nunciation.py:296 y scripts/audio_pronunciation.py:385
AudioClassifier Head tuyo audio: clasificador del embedding	Clase: scripts/train_audio_classifier.py:95 Entrena: scripts/train_audio_classifier.py:168	scripts/train_audio_classifier.py:272 → models/audio_classifier.pth (scripts/train_audio_classifier.py:16)	server/main.py:524	server/main.py:908 (softmax en 907)

También se usa **MediaPipe Face Landmarker** (modelo externo) directamente en el navegador, para las praxias de los ejercicios: web/src/screens/EjerciciosScreen.jsx. No pasa por el servidor.

Qué pasa cuando el niño dice una palabra frente a la cámara

1

La app manda el clip

Código / procedimiento

web `web/src/screens/HomeScreen.jsx:451` web `web/src/api.js:263`

```
predict(baseUrl, videoBlob, filename, onRetrying, audioBlob, palabra)
```

La pantalla de práctica graba video y audio y llama a predict(), que hace un POST a /predict del servidor.

↓

2

Endpoint /predict

Código / procedimiento

servidor `server/main.py:1259`

```
predict(file, audio, palabra)
```

Recibe el video y el audio, guarda el audio en un archivo temporal y manda todo al hilo de la GPU con _run_prediction_pipeline. Si solo llegó audio (modo «solo voz»), usa _run_audio_only_pipeline.

↓

3 HRNet saca los puntos de la cara

Modelo externo ya entrenado

servidor `server/main.py:1058`

```
detect_landmarks_batch(landmarker, frames)
```

Detecta los 68 puntos en cada frame (de a 4 frames). De ahí se usan los 68 y se normalizan con los ojos igual que en el entrenamiento.

↓

4 Se comprueba que la palabra tenga modelo de video (cara completa)

Código / procedimiento

servidor `server/main.py:1049`

```
modelo_video = 'cara_completa'
```

Si la palabra pedida está en el modelo de cara completa y se ven los ojos, se usan los 68 puntos normalizados con los ojos (408 números por frame). Si la palabra no está (perro) se evalúa solo con el audio, y si no se ven los ojos se pide repetir la toma.

↓

5 El modelo de video clasifica

Tu modelo entrenado

servidor `server/main.py:1131`

```
logits = model(sequence, src_key_padding_mask=mask)
```

LipReadingConformer recibe la secuencia (1, 60, 408) de cara completa y devuelve una probabilidad por clase (54 clases). Se toma la clase más probable y su confianza.

↓

6 wav2vec2 + GOP puntúa cada sonido

Modelo externo ya entrenado

servidor `server/main.py:878` audio `scripts/audio_pronunciation.py:296`

```
_score_audio_pronunciation(audio_path, palabra)
```

Alinea el audio con los fonemas esperados de la palabra y puntúa cada uno (GOP). Marca como «sospechosos» los que suenan dudosos y revisa si se dijo otra palabra.

↓

7 El clasificador de audio decide si suena correcto

Tu modelo entrenado

servidor `server/main.py:908` embedding `server/main.py:920`

```
_classify_audio(audio_path, palabra)
```

Saca el embedding de 1024 números con wav2vec2, lo normaliza con la media y desviación guardadas en el checkpoint, y AudioClassifierHead devuelve una probabilidad por clase (softmax). Solo se mira las clases de la palabra pedida.

↓

8 Se combinan las dos señales en un veredicto

Código / procedimiento

servidor `server/main.py:1178 a 1175`

```
acoustic_match · visual_match · audio_confirma
```

Si el audio confirma la palabra (clasificador correcto con buena confianza, o confianza media y el GOP de acuerdo), el veredicto es «bien», salvo que el video esté muy seguro de que fue la misma palabra dicha mal. Si no, se combinan las dos señales y sale «bien», «a practicar» o «no concluyente».

Otros lugares donde se usan los modelos

Para qué	Endpoint (servidor)	Qué modelos usa	Quién lo llama (web)

Palabra con cámara	server/main.py:1259 /predict	HRNet + LipReadingConformer (cara completa) + wav2vec2 + AudioClassifierHead	web/src/screens/HomeScreen.jsx:451 · api.js:263
Frase solo con voz (prueba y práctica)	server/main.py:1328 /predict_frase	wav2vec2 (GOP) + AudioClassifierHead	web/src/screens/LearningPathScreen.jsx:909 · web/src/screens/DiagnosticoScreen.jsx:137 · api.js:228
Ejercicios de voz (pa-ta-ka, trabalenguas)	server/main.py:1469 /ejercicio_voz	wav2vec2 (reconocimiento libre y GOP)	web/src/screens/EjerciciosScreen.jsx:524 · api.js:284
Praxias de cara (sonrisa, beso, mejillas…)	No usa el servidor	MediaPipe Face Landmarker, en el navegador	web/src/screens/EjerciciosScreen.jsx

Preguntas que te pueden hacer

¿Cuáles de los modelos entrenaste tú?

Dos: LipReadingConformer (video) y AudioClassifierHead (audio). HRNet/FAN y wav2vec2 ya venían entrenados y solo se usan; el GOP es un procedimiento sobre wav2vec2, no un modelo.

¿Dónde se carga el modelo?

Una sola vez, al arrancar el servidor, en load_everything (server/main.py:445). Por eso la primera petición no tiene que cargar nada.

¿Qué archivos son los modelos?

data/modelos_cara_completa/best_cara_completa.pth (video) y models/audio_classifier.pth (audio). Cada uno guarda sus pesos, los nombres de las clases y, el de video, el tamaño de entrada.

¿Cuál pesa más en el veredicto?

El audio: mide ~97 % de acierto contra ~90 % del video de cara completa. El video es la señal de apoyo y solo bloquea cuando está muy seguro de que se dijo la misma palabra mal.

¿Y si el servidor no tiene el modelo de audio?

Sigue funcionando solo con el video: la carga del audio está en un try/except y avisa en consola.